Fair share scheduling (FSS) supports scheduling across related processes and threads. It enables a system to ensure fairness across groups of processes by restricting each group to a certain subset of system resources. In the UNIX environment, for example, FSS was developed specifically to "give a prespecified rate of system resources ... to a related set of users."

In deadline scheduling, certain processes are scheduled to be completed by a specific time or deadline. These processes may have high value if delivered on time and be worthless otherwise. Deadline scheduling can be difficult to implement.

Real-time scheduling must repeatedly meet processes' deadlines as they execute. Soft real-time scheduling ensures that real-time processes are dispatched before other processes in the system. Hard real-time scheduling guarantees that a process's deadlines are met. Hard real-time processes must meet their deadlines; failing to do so could be catastrophic, resulting in invalid work, system failure or even harm to the system's users. The rate-monotonic (RM) algorithm is a static scheduling priority-based round-robin algorithm in which priorities are increase (monotonically) with the rate at which a process must be scheduled. The earliest-deadline-first (EDF) is a preemptive scheduling algorithm that favors the process with the earliest deadline. The minimum-laxity-first algorithm is similar to EDF. but bases priority on a process's laxity, which measures the difference between the time a process requires to complete and the time remaining until that its deadline.

The operating system may dispatch a process's threads individually or it may schedule a multithreaded process as a unit, requiring user-level libraries to schedule their threads. System designers must determine how to allocate quanta to threads, in what order, and what priorities to assign in scheduling threads within a process. In Java, each thread is assigned a priority in the range 1-10. Depending on the platform, Java implements threads in either user or kernel space (see Section 4.6, Threading Models). When implementing threads in user space, the Java runtime relies on timeslicing to perform preemptive thread scheduling. The Java thread scheduler ensures that the highest-priority thread in the Java virtual machine is running at all times. If there are multiple threads at the same priority level, those threads execute using round-robin. A thread can call the `yield` method to give other threads of an equal priority a chance to run.

Key Terms

adaptive mechanism—Control entity that adjusts a system in response to its changing behavior.

admission scheduling—See high-level scheduling.

aging of priorities—Increasing a process's priority gradually, based on how long it has been in the system.

asynchronous real-time process—Real-time process that executes in response to events.

batch process—Process that executes without user interaction.

deadline rate-monotonic scheduling—Scheduling policy in real-time systems that meets a periodic process's deadline that does not equal its period.

deadline scheduling—Scheduling policy that assigns priority based on processes' completion deadlines to ensure that processes complete on time.

degree of multiprogramming—Total number of processes in main memory at a given time.

dispatcher—Entity that assigns a processor to a process.

dispatching—Act of assigning a processor to a process.

dynamic real-time scheduling algorithm—Scheduling algorithm that uses deadlines to assign priorities to processes throughout execution.

earliest-deadline-first (EDF)—Scheduling policy that gives a processor to the process with the closest deadline.

fairness—Property of a scheduling algorithm that treats all processes equally.

fair share group—Group of processes that receives a percentage of the processor time under a fair share scheduling (FSS) policy.

fair share scheduling (FSS)—Scheduling policy developed for AT&T's UNIX system that places processes in groups and assigns these groups a percentage of processor time.

first-in-first-out (FIFO)—Nonpreemptive scheduling policy that dispatches processes according to their arrival time in the ready queue.

job scheduling—See high-level scheduling.

hard real-time scheduling—Scheduling policy that ensures processes meet their deadlines.

highest-response-ratio-next (HRRN)—Scheduling policy that assigns priority based on a process's service time and the amount of time the process has been waiting.

high-level scheduling—Determining which jobs a system allows to compete actively for system resources.

interactive process—Process that requires user input as it executes.

intermediate-level scheduling—Determining which processes may enter the low-level scheduler to compete for a processor.

I/O-bound—Process that tends to use a processor for a short time before generating an I/O request and relinquishing a processor.

latency (process scheduling)—Time a task spends in a system before it is serviced.

laxity—Value determined by subtracting the sum of the current time and a process's remaining execution time from the process's deadline. This value decreases as a process nears its deadline.

long-term scheduling—See high-level scheduling.

low-level scheduling—Determining which process will gain control of a processor.

minimum-laxity-first—Scheduling policy that assigns higher priority to processes that will finish with minimal processor usage.

multilevel feedback queue—Process scheduling structure that groups processes of the same priority in the same round-robin queue. Processor-bound processes are placed in lower-priority queues because they are typically batch processes that do not require fast response times. I/O-bound processes, which exit the system quickly due to I/O, remain in high-priority queues. These processes often correspond to interactive processes that should experience fast response times.

nonpreemptive scheduling—Scheduling policy that does not allow the system to remove a processor from a process until that process voluntarily relinquishes its processor or runs to completion.

periodic real-time process—Real-time process that performs computation at a regular time interval.

predictability—Goal of a scheduling algorithm that ensures that a process takes the same amount of time to execute regardless of the system load.

preemptive scheduling—Scheduling policy that allows the system to remove a processor from a process.

priority—Measure of a process's or thread's importance used to determine the order and duration of execution.

processor-bound—Process that consumes its quantum when executing. These processes tend to be calculation intensive and issue few, if any, I/O requests.

processor scheduling discipline—See processor scheduling policy.

processor scheduling policy—Strategy used by a system to determine when and for how long to assign processors to processes.

purchase priority—to pay to receive higher priority in a system.

quantum—Amount of time that a process is allowed to run on a processor before the process is preempted.

rate-monotonic (RM) scheduling—Real-time scheduling policy that sets priority to a value that is proportional to the rate at which the process must be dispatched.

real-time scheduling—Scheduling policy that bases priority on timing constraints.

round-robin (RR) scheduling—Scheduling policy that permits each *ready* process to execute for at most one quantum per round. After the last process in the queue has executed once, the scheduler begins a new round by scheduling the first process in the queue,

scalability (scheduler)—Characteristic of a scheduler that ensures system performance degrades gracefully under heavy loads.

selfish round-robin (SRR) scheduling—Variant of round-robin scheduling in which processes age at different rates. Processes that enter the system are placed in a holding queue, where they wait until their priority is high enough for them to be placed in the active queue, in which processes compete for processor time.

shortest-process-first (SPF) scheduling—Nonpreemptive scheduling algorithm in which the scheduler selects a process with the smallest estimated run-time-to-completion and runs the process to completion.

shortest-remaining-time (SRT) scheduling—Preemptive version of SPF in which the scheduler selects a process with the smallest estimated remaining run-time-to-completion.

soft real-time scheduling—Scheduling policy that guarantees that real-time processes are scheduled with higher priority than non-real-time processes.

static real-time scheduling algorithm—Scheduling algorithm that uses timing constraints to assign fixed priorities to processes before execution.

throughput—Number of processes that complete per unit time.

time slice— See quantum.

timeslicing—Scheduling each process to execute for at most one quantum before preemption.

timing constraint—Time period during which a process (or subset of a process's instructions) must complete.

Exercises

8.1 Distinguish among the following three levels of schedulers.

a. high-level scheduler

b. intermediate-level scheduler

c. dispatcher

8.2 Which level of scheduler should make a decision on each of the following questions?

a. Which ready process should be assigned a processor when one becomes available?

b. Which of a series of waiting batch processes that have been spooled to disk should next be initiated?

c. Which processes should be temporarily suspended to relieve a short-term burden on the processor?

d. Which temporarily suspended process that is known to be I/O bound should be activated to balance the multiprogramming mix?

8.3 Distinguish between a scheduling policy and a scheduling mechanism.

8.4 The following are common scheduling objectives.

a. to be fair

b. to maximize throughput

c. to maximize the number of interactive users receiving acceptable response times

d. to be predictable

e. to minimize overhead

f. to balance resource utilization

g. to achieve a balance between response and utilization

h. to avoid indefinite postponement

i. to obey priorities

j. to give preference to processes that hold key resources

k. to give a lower grade of service to high overhead processes

l. to degrade gracefully under heavy loads

Which of the preceding objectives most directly applies to each of the following?

i. If a user has been waiting for an excessive amount of time, favor that user.

ii. The user who runs a payroll job for a 1000-employee company expects the job to take about the same amount of time each week.

iii. The system should admit processes to create a mix that will keep most devices busy.

iv. The system should favor important processes.

v. An important process arrives but cannot proceed because an unimportant process is holding the resources the important process needs.

vi. During peak periods, the system should not collapse from the overhead it takes to manage a large number of processes.

vii. The system should favor I/O-bound processes.

viii. Context switches should execute as quickly as possible.

8.5 The following are common scheduling criteria.

a. I/O boundedness of a process

b. processor boundedness of a process

c. whether the process is batch or interactive

d. urgency of a fast response

e. process priority

f. frequency of a process is being preempted by higher-priority processes

g. priorities of processes waiting for resources held by other processes

h. accumulated waiting time

i. accumulated execution time

j. estimated run-time-to-completion.

For each of the following, indicate which of the preceding scheduling criteria is most appropriate.

i. In a real-time spacecraft monitoring system, the computer must respond immediately to signals received from the spacecraft.

ii. Although a process has been receiving occasional service, it is making only nominal progress.

iii. How often does the process voluntarily give up the processor for I/O before its quantum expires?

iv. Is the user present and expecting fast interactive response times, or is the user absent?

v. One goal of processor scheduling is to minimize average waiting times.

vi. Processes holding resources in demand by other processes should have higher priorities.

vii. Nearly complete processes should have higher priorities.

8.6 State which of the following are true and which false. Justify your answers.

a. A process scheduling discipline is preemptive if the processor cannot be forcibly removed from a process.

b. Real-time systems generally use preemptive processor scheduling.

c. Timesharing systems generally use nonpreemptive processor scheduling.

d. Turnaround times are more predictable in preemptive than in nonpreemptive systems.

e. One weakness of priority schemes is that the system will faithfully honor the priorities, but the priorities themselves may not be meaningful.

8.7 Why should processes be prohibited from setting the interrupting clock?

8.8 State which of the following refer to "static priorities" and which to "dynamic priorities."

a. are easier to implement

b. require less runtime overhead

c. are more responsive to changes in a process's environment

d. require more careful deliberation over the initial priority value chosen

8.9 Give several reasons why deadline scheduling is complex.

8.10 Give an example showing why FIFO is not an appropriate processor scheduling scheme for interactive users.

8.11 Using the example from the previous problem, show why round-robin is a better scheme for interactive users.

8.12 Determining the quantum is a complex and critical task. Assume that the average context-switching time between processes is *s,* and the average amount of time an I/O-bound process uses before generating an I/O request is t ($t >> s$). Discuss the effect of each of the following quantum settings, q.

a. q is slightly greater than zero

b. $q = s$

c. $s < q < t$

d. $q = t$

e. $q > t$

f. q is an extremely large number

8.13 Discuss the effect of each of the following methods of assigning q.

a. q fixed and identical for all users

b. q fixed and unique to each process

c. q variable and identical for all processes

d. q variable and unique to each process

i. Arrange the schemes above in order from lowest to highest runtime overhead.

ii. Arrange the schemes in order from least to most responsive to variations in individual processes and system load.

iii. Relate your answers in (i) and (ii) to one another.

8.14 State why each of the following is incorrect.

a. SPF never has a higher throughput than SRT.

b. SPF is fair.

c. The shorter the process, the better the service it should receive.

d. Because SPF gives preference to short processes, it is useful in timesharing.

8.15 State some weaknesses in SRT. How would you modify the scheme to get better performance?

8.16 Answer each of the following questions about Brinch Hansen's HRRN strategy.

a. How does HRRN prevent indefinite postponement?

b. How does HRRN decrease the favoritism shown by other strategies to short new processes?

c. Suppose two processes have been waiting for about the same time. Are their priorities about the same? Explain your answer.

8.17 Show how multilevel feedback queues accomplish each of the following scheduling goals.

a. favor short processes.

b. favor I/O-bound processes to improve I/O device utilization.

c. determine the nature of a process as quickly as possible and schedule the process accordingly.

8.18 One heuristic often used by processor schedulers is that a process's past behavior is a good indicator of its future behavior. Give several examples of situations in which processor schedulers following this heuristic would make bad decisions.

8.19 An operating systems designer has proposed a multilevel feedback queuing network in which there are five levels. The quantum at the first level is 0.5 seconds. Each lower level has a quantum twice the size of the quantum at the previous level. A process cannot be preempted until its quantum expires. The system runs both batch and interactive processes, and these consist of both processor-bound and I/O-bound processes.

a. Why is this scheme deficient?

b. What minimal changes would you propose to make the scheme more acceptable for its intended process mix?

8.20 The SPF strategy can be proven to be optimal in the sense that it minimizes average response times. In this problem, you will demonstrate this result empirically by examining all possible orderings for a given set of five processes. In the next problem, you will actually prove the result. Suppose five different processes are waiting to be processed, and that they require 1,2, 3, 4 and 5 time units, respectively. Write a program that produces all possible permutations of the five processes (5! = 120) and calculates the average waiting time for each permutation. Sort these into lowest to highest average waiting time order and display each average time side by side with the permutation of the processes. Comment on the results.

8.21 Prove that the SPF strategy is optimal in the sense that it minimizes average response times. *[Hint:* Consider a list of processes each with an indicated duration. Pick any two processes arbitrarily. Assuming that one is larger than the other, show the effect that placing the smaller process ahead of the longer one has on the waiting time of each process. Draw an appropriate conclusion.]

8.22 Two common goals of scheduling policies are to minimize response times and to maximize resource utilization.

a. Indicate how these goals are at odds with one another.

b. Analyze each of the scheduling policies presented in this chapter from these two perspectives. Which are biased toward minimizing user response times? Which are biased toward maximizing resource utilization?

c. Develop a new scheduling policy that enables a system to be tuned to achieve a good balance between these conflicting objectives.

8.23 In the selfish round-robin scheme, the priority of a process residing in the holding queue increases at a rate a until the priority is as high as that of processes in the active queue, at which point the process enters the active queue and its priority continues to increase, but now at the rate b. Earlier we stated that $b \leq a$. Discuss the behavior of an SRR system if $b > a$.

8.24 How does a fair share scheduler differ in operation from a conventional process scheduler?

8.25 In a uniprocessor system with n processes, how many different ways are there to schedule an execution path?

8.26 Suppose a system has a multilevel feedback queue scheduler implemented. How could one adapt this scheduler to form the following schedulers?

a. FCFS

b. round-robin

8.27 Compare and contrast EDF and SPF.

8.28 When are static real-time scheduling algorithms more appropriate than dynamic real-time scheduling algorithms?

8.29 Before Sun implemented its current Solaris operating system, Sun's UNIX system,[70] primarily used for workstations, performed process scheduling by assigning base priorities (a high of -20 to a low of +20 with a median of 0), and making priority adjustments. Priority adjustments were computed in response to changing system conditions. These were added to base priorities to compute current priorities; the process with the highest current priority was dispatched first.

The priority adjustment was strongly biased in favor of processes that had recently used relatively little processor time. The scheduling algorithm "forgot" processor usage quickly to give the benefit of the doubt to processes with changing behavior. The algorithm "forgot" 90 percent of recent processor usage in 5 * n seconds; n is the average number of runnable processes in the last minute.[71] Consider this process scheduling algorithm when answering each of the following questions.

a. Are processor-bound processes favored more when the system is heavily loaded or when it is lightly loaded?

b. How will an I/O-bound process perform immediately after an I/O completion?

c. Can processor-bound processes suffer indefinite postponement?

d. As the system load increases, what effect will processor-bound processes have on interactive response times?

e. How does this algorithm respond to changes in a process's behavior from processor bound to I/O bound or vice versa?

8.30 The VAX/VMS[72] operating system ran a wide variety of computers, from small to large. In VAX/VMS, process priorities range from 0-31, with 31 being the highest priority assigned to critical real-time processes. Normal processes receive priorities in the range 0-15; real-time processes receive priorities in the range 16-31. The priorities of real-time processes normally remain constant; no priority adjustments are applied. Real-time processes continue executing (without suffering quantum expirations) until they are preempted by processes with higher or equal priorities, or until they enter various *wait* states.

Other processes are scheduled as normal processes with priorities 0-15. These include interactive and batch processes, among others. Normal process priorities do vary. Their base priorities normally remain fixed, but they receive dynamic priority adjustments to give preference to I/O-bound over processor-bound processes. A normal process retains the processor

until it is preempted by a more important process, until it enters a special state such as an event wait, or until its quantum expires. Processes with the same current priorities are scheduled round-robin.

Normal processes receive priority adjustments of 0 to 6 above their base priorities. Such positive increments occur, for example, when requested resources are made available, or when a condition on which a process is waiting is signaled. I/O-bound processes tend to get positive adjustments in priority, while processor-bound processes tend to have current priorities near their base priorities. Answer each of the following questions with regard to the VAX/VMS process scheduling mechanisms.

a. Why are the priorities of real-time processes normally kept constant?
b. Why are real-time processes not susceptible to quantum expirations?
c. Which normal processes, I/O bound or processor bound, generally receive preference?
d. Under what circumstances can a real-time process lose the processor?
e. Under what circumstances can a normal process lose the processor?
f. How are normal processes with the same priorities scheduled?
g. Under what circumstances do normal processes receive priority adjustments?

8.31 Prepare a research paper that compares and contrasts how Windows XP, Linux, and Mac OS X schedule processes.

8.32 Although the operating system is typically responsible for scheduling processes and threads, some systems allow user processes to make scheduling decisions. Research how this is done for process scheduling in exokernel operating systems (see www.pdos.lcs.mit.edu/pubs.html#Exokernels) and for thread scheduling in user-level threads and scheduler activations.

8.33 Research how real-time scheduling is implemented in embedded systems.

8.34 Research the scheduling policies and mechanisms that are commonly used in database systems.

Suggested Simulations

8.35 One solution to the problem of avoiding indefinite postponement of processes is aging, in which the priorities of waiting processes increase the longer they wait. Develop several aging algorithms and write simulation programs to examine their relative performance. For example, a process's priority may be made to increase linearly with time. For each of the aging algorithms you choose, determine how well waiting processes are treated relative to high-priority arriving processes.

8.36 One problem with preemptive scheduling is that the context-switching overhead may tend to dominate if the system is not carefully tuned. Write a simulation program that determines the effects on system performance of the ratio of context-switching overhead to typical quantum time. Discuss the factors that determine context-switching overhead and the factors that determine the typical quantum time. Indicate how achieving a proper balance between these (i.e., tuning the system) can dramatically affect system performance.

Recommended Reading

Coffman and Kleinrock discuss popular scheduling policies and indicate how users who know which scheduling policy the system employs can actually achieve better performance by taking appropriate measures.[73] Ruschitzka and Fabry give a classification of scheduling algorithms, and they formalize the notion of priority.[74] Today's schedulers typically combine several techniques described in this chapter to meet particular system objectives. Recent research in the field of processor scheduling focuses on optimizing performance in settings such as Web servers, real-time systems and embedded devices. Stankovic et al. present a discussion of real-time scheduling concerns,[75] Pop et. al present scheduling concerns in embedded systems[76] and Bender et al. discuss scheduling techniques for Web server transactions.[77] SMART (Scheduler for Multimedia And Real-Time applications), which combines real-time and traditional scheduling techniques to improve multimedia performance, was developed in response to increasing popularity of multimedia applications on workstations and personal computers.[78]

Dertouzos demonstrated that when all processes can meet their deadlines regardless of the order in which they are executed, it is optimal to schedule processes with the earliest deadlines first.[79] However, when the system becomes overloaded, the optimal scheduling policy cannot be implemented without allocating significant processor time to the scheduler. Recent research focuses on the speed[80, 81] or number[82] of processors the system must allocate to the scheduler so that it meets its deadlines. The bibliography for this chapter is located on our Web site at www.deitel.com/books/osBe/Bibliography.pdf.

Works Cited

1. Ibaraki, T.; Abdel-Wahab, H. M; and T. Kameda, "Design of Minimum-Cost Deadlock-Free Systems," *Journal of the ACM.,* Vol. 30, No. 4 October 1983, p. 750.
2. Coffman, E. G., Jr., and L. Kleinrock, "Computer Scheduling Methods and Their Countermeasures," *Proceedings of AFIPS,* S/CC, Vol. 32,1968, pp. 11-21.
3. Ruschitzka, M., and R. S. Fabry, "A Unifying Approach to Scheduling," *Communications oftheACM.Vol.* 20, No. 7, July 1977, pp. 469-477.
4. Abbot, C, "Intervention Schedules for Real-Time Programming," *IEEE Transactions on Software Engineering,* Vol. SE-10, No. 3, May 1984, pp. 268-274.
5. Ramamritham, K., and J. A. Stanovic, "Dynamic Task Scheduling in Hard Real-Time Distributed Systems," *IEEE Software,* Vol. 1. No. 3, July 1984, pp. 65-75.
6. Volz, R. A., and T. N. Mudge, "Instruction Level Timing Mechanisms for Accurate Real-Time Task Scheduling," *IEEE Transactions on Computers,* Vol. C-36, No. 8, August 1987, pp. 988-993.
7. Potkonjak, M., and W. Wolf, "A Methodology and Algorithms for the Design of Hard Real-Time Multitasking ASICs," *ACM Transactions on Design Automation of Electronic Systems (TODAES)* Vol, 4. No. 4, October 1999.
8. Kleinrock, L., "A Continuum of Time-Sharing Scheduling Algorithms," *Proceedings of AFIPS, SJCC,* 1970. pp. 453-458.
9. "sched.h—Execution Scheduling (REALTIME)," The Open Group Base Specifications Issue 6, IEEE Std. 1003.1, 2003 edition, <www.opengroup.org/onlinepubs/007904975/basedefs/sched.h.html>.
10. Kleinrock, L., "A Continuum of Time-Sharing Scheduling Algorithms," *Proceedings of AFIPS, SJCC.* 1970, pp. 453-458.
11. Potier, D.; Gelenbe, E.; and J. Lenfant, "Adaptive Allocation of Central Processing Unit Quanta," *Journal of the ACM,* Vol. 23, No. 1. January 1976, pp. 97-102.
12. Linux source code, version 2.6.0-test3, sched.c, lines 62-135 <miller.cs.wm.edu/lxr3.linux/http/source/kernel/sched.c?v=2.6.0-test3>.
13. Solomon, D., and M. Russinovich, *Inside Windows 2000,* 3rd ed., Redmond: Microsoft Press. 2000, pp. 338,347.
14. Bruno, X; Coffman, E. G, Jr.; and R. Sethi, "Scheduling Independent Tasks to Reduce Mean Finishing Time," *Communications of the ACM,* Vol. 17, No. 7, July 1974, pp. 382-387.
15. Deitel, H. M., "Absentee Computations in a Multiple Access Computer System," M.I.T. Project MAC MAC-TR-52. Advanced Research Projects Agency, Department of Defense. 1968.
16. Brinch Hansen, P., "Short-Term Scheduling in Multiprogram ming Systems," *Third ACM Symposium on Operating Systems Principles,* Stanford University, October 1971, pp. 103-105.
17. Kleinrock, L., "A Continuum of Time-Sharing Scheduling Algorithms," *Proceedings ofAFIPS, SJCC,* 1970, pp. 453-458.
18. Blevins, P. R., and C. V. Ramamoorthy, 'Aspects of a Dynamically Adaptive Operating System," *IEEE Transactions on Computers,* Vol. 25, No. 7, July 1976, pp. 713-725.
19. Potier, D.; Gelenbe, E.; and J. Lenfant, "Adaptive Allocation of Central Processing Unit Quanta," *Journal of the ACM,* Vol. 23. No. 1, January 1976, pp. 97-102.
20. Newbury, J. P.. "Immediate Turnaround—An Elusive Goal." *Software Practice and Experience,* Vol. 12, No. 10, October 1982. pp. 897-906.
21. Henry, G. X, "The Fair Share Scheduler," *Bell Systems Technical Journal,* Vol. 63, No. 8, Part 2, October 1984, pp. 1845-1857.
22. Woodside, C. M., "Controllability of Computer Performance Tradeoffs Obtained Using Controlled-Share Queue Schedulers." *IEEE Transactions on Software Engineering,* Vol. SE-12, No. 10. October 1986, pp. 1041-1048.
23. Kay, X, and P. Lauder, "A Fair Share Scheduler," *Communications oftheACM.Vol.* 31, No. 1, January 1988, pp. 44-55.
24. Henry, G J, "The Fair Share Scheduler," *Bell Systems Technical Journal,* Vol. 63, No. 8, Part 2, October 1984, p. 1846.
25. "Chapter 9, Fair Share Scheduler," *Solaris 9 System Administrator Collection,* November 11, 2003, <docs.sun.com/db/doc/806-4076/6jd6amqqo?a=view>.
26. Henry, G X, "The Fair Share Scheduler," *Bell Systems Technical Journal,Vol.* 63, No. 8, Part 2, October 1984, p. 1848.
27. Henry, G X, "The Fair Share Scheduler," *Bell Systems Technical JournafVol.* 63, No. 8, Part 2, October 1984, p. 1847.
28. Henry, G. X, "The Fair Share Scheduler," *Bell Systems Technical Journal,* Vol. 63, No. 8, Part 2, October 1984, p. 1846.
29. Nielsen, N. R., "The Allocation of Computing Resources—Is Pricing the Answer?" *Communications of the ACM,* Vol. 13, No. 8, August 1970, pp. 467-474.

30. McKell, L. X; Hansen, J. V.; and L. E. Heitger, "Charging for Computer Resources," *ACM Computing Surveys,* Vol. 11, No. 2, June 1979, pp. 105-120.
31. Kleijnen, A. J. V., "Principles of Computer Charging in a University-Type Organization," *Communications of the ACM,* Vol. 26, No. 11, November 1983, pp. 926-932.
32. Dertouzos, M. L., "Control Robotics: The Procedural Control of Physical Processes," *Proc. IFIP Congress,* 1974, pp. 807-813.
33. Kalyanasundaram, B., and K., Pruhs, "Speed Is As Powerful As Clairvoyance," *Journal of the ACM (JACM),* Vol. 47, No. 4, July, 2002, pp. 617-643.
34. Lam, T, and K. To, "Performance Guarantee for Online Deadline Scheduling in the Presence of Overload," *Proceedings of the TWELFTH ANNUal ACM-SIAM Symposium on Discrete Algorithms,* January 7-9,2001, pp. 755-764.
35. Koo, C; Lam, T; Ngan, T.; and K. To, "Extra Processors Versus Future Information in Optimal Deadline Scheduling," *Proceedings of the Fourteenth Annual ACM Symposium on Parallel Algorithms and Architectures,* 2002, pp. 133-142.
36. Stankovic, J., "Real-Time and Embedded Systems," *ACM Computing Surveys,Vol.* 28, No. 1, March 1996, pp. 205-208.
37- Xu, J., and D. L. Parnas, "On Satisfying Timing Constraints in Hard-Real-Time Systems," *Proceedings of the Conference on Software for Critical Systems,* New Orleans, Louisiana, 1991, pp. 132-146.
38. Stewart, D., and P. Khosla, "Real-Time Scheduling of Dynamically Reconfigurable Systems," *Proceedings of the IEEE International Conference on Systems Engineering,* Dayton, Ohio, August 1991, pp. 139-142.
39. van Beneden, B., "Comp.realtime: Frequently Asked Questions (FAQs) (Version 3.6)," May 16, 2002 <www.faqs.org/faqs/realtime-computing/faq/>.
40. MITRE Corporation, "MITRE - About Us - MITRE History - Semi-Automatic Ground Environment (SAGE)," January 7, 2003 <www.mitre.org/about/sage.html>.
41. Edwards, P., "SAGE," *The Closed World,* Cambridge, MA: MIT Press, 1996, <www.si.urrrich.edu/~pne/PDF/cw.ch3.pdf>.
42. Edwards, P., "SAGE," *The Closed World,* Cambridge, MA: MIT Press, 1996, <www.si.umich.edu/~pne/PDF/cw.ch3.pdf>.
43. MITRE Corporation, "MITRE-About Us-MITRE History-Semi-Automatic Ground Environment (SAGE)," January 7, 2003 <www.mitre.org/about/sage.html>.
44. Edwards, P., "SAGE," *The Closed World,* Cambridge, MA: MIT Press, 1996, <www.si.umich.edu/~pne/PDF/cw.ch3.pdf>.
45. MITRE Corporation. "MITRE-About Us-MITRE History-Semi-Automatic Ground Environment (SAGE)," January 7, 2003 <www.mitre.org/about/sage.html>.
46. Edwards, P., "SAGE," *The Closed World,* Cambridge, MA: MIT Press, 1996, <www.si.umich.edu/~pne/PDF/cw.ch3.pdf>.
47. QNX Software Systems Ltd., "The Philosophy of QNX Neutrino," <www.qnx.com/developer/docs/momentics621_docs/neutrino/sys_arch/intro.html>.
48. Wind River Systems, Inc., "VxWorks 5.x." <www.windriver.com/products/vxworks5/vxworks5x_ds.pdf>.
49. Dedicated Systems Experts, "Comparison between QNX RTOS V6.1, VxWorks AE 1.1, and Windows CE .NET," June 21, 2001 <www.eon-trade.com/data/QNX/QNX61_VXAE_CE.pdf>.
50. QNX Software Systems Ltd., "QNX Supported Hardware," <www.qnx.com/support/sd_hardware/platform/processors.html>.
51. Timmerman, M., "RTOS Evaluation Project Latest News," *Dedicated Systems Magazine,* 1999, <www.omimo.be/magazine/99ql/1999ql_p009.pdf>.
52. Wind River Systems, Inc.,"VxWorks 5.x.," <www.windri ver.com/products/vxworks5/vxworks5x_ds.pdf>.
53. Microsoft Corporation, "Windows CE .NET Home Page," <www.microsoft.com/windowsce/default .asp>.
54. Radisys Corporation, "RadiSys: Microware OS-9," <www.radisys.com/oem_products/op-os9.cfm?MS=Microware%20Enhanced%200S-9%20Solution>.
55. Enea Embedded Technology, "Welcome to Enea Embedded Technology," <www.ose.com>.
56. Real Time Linux Foundation, Inc., "Welcome to the Real Time Linux Foundation Web Site," <www.realtimelinuxfoundation.org>.
57. Etsion, Y.; Tsafrir, D.; and D. Feiteelson, "Effects of Clock Resolution on the Scheduling of Interactive and Soft Real-Time Processes," *SIGMETRICS'03,* June 10-14,2003, pp. 172-183.
58. Xu, J., Parnas, D. L., "On Satisfying Timing Constraints in Hard-Real-Time Systems," *Proceedings of the Conference on Software for Critical Systems,* New Orleans, Louisiana, 1991, pp. 132-146.
59. Stewart, D., and P. Khosla, "Real-Time Scheduling of Dynamically Reconfigurable Systems," *Proceedings of the IEEE International Conference on Systems Engineering,* Dayton, Ohio, August 1991, pp. 139-142.
60. Xu, J., and D. L. Parnas, "On Satisfying Timing Constraints in Hard-Real-Time Systems," *Proceedings of the Conference on Software for Critical Systems,* New Orleans, Louisiana, 1991, pp. 132-146.
61. Stewart, D., and P. Khosla, "Real-Time Scheduling of Dynamically Reconfigurable Systems," *Proceedings of the IEEE International Conference on Systems Engineering,* Dayton, Ohio, August 1991, pp. 139-142.
62. Potkonjak, M., W Wolf, "A Methodology and Algorithms for the Design of Hard Real-Time Multitasking ASICs," *ACM Transactions on Design Automation of Electronic Systems (TODAES),* Vol. 4, No. 4, October 1999.
63. Xu, X, and D. L. Parnas, "On Satisfying Timing Constraints in Hard-Real-Time Systems," *Proceedings of the Conference on Software for Critical Systems,* New Orleans, Louisiana, 1991, pp. 132-146.
64. Dertouzos, M. L., "Control Robotics: The Procedural Control of Physical Processes," *Information* Processing, Vol. 74,1974.

65. Stewart, D., and P. Khosla, "Real-Time Scheduling of Dynamically Reconfigurable Systems," *Proceedings of the IEEE International Conference on Systems Engineerings* Dayton, Ohio, August 1991, pp. 139-142.

66. Stewart, D., and P. Khosla, "Real-Time Scheduling of Dynamically Reconfigurable Systems," *Proceedings of the IEEE International Conference on Systems Engineering,* Dayton, Ohio, August 1991, pp. 139-142.

67. "A Look at the JVM and Thread Behavior," June 28, 1999, <www.javaworld.com/javaworld/javaqa/1999-07/04-qa-jvmthreads.html>.

68. Austin, C, "Java Technology on the Linux Platform: A Guide to Getting Started," October 2000, <developer.java.sun.com/developer/technicalArticles/Programming/linux/>.

69. Holub, A., "Programming Java Threads in the Real World, Part 1," *JavaWorld,* September 1998, <www.javaworld.com/java¬world/jw-09-1998/jw-09-threads.html>.

70. Courington, W., *The UNIX System: A Sun Technical Report,* Mountain View, CA: Sun Microsystems, Inc., 1985.

71. Courington, W., *The UNIX System: A Sun Technical Report,* Mountain View, CA: Sun Microsystems, Inc., 1985.

72. Kenah, L. J.; Goldenberg, R. E.; and S. F. Bate, *VAX/VMS Internals and Data Structures: Version 4.4,* Bedford, MA: Digital Equipment Corporation, 1988.

73. Coffman, E. G, Jr., and L. Kleinrock, "Computer Scheduling Methods and Their Countermeasures," *Proceedings of AFIPS, SJCC,* Vol. 32,1968, pp. 11-21.

74. Ruschitzka, M., and R. S. Fabry, "A Unifying Approach to Scheduling," *Communications of the ACM,* Vol. 20, No. 7, July 1977, pp. 469-477.

75. Stankovic, J. A.; Spuri, M.; Natale, M. D.; and G. C. Buttazzo. "Implications of Classical Scheduling Results for Real-Time Systems," *IEEE Computer,Vo\.* 28, No. 6,1995, pp. 16-25.

76. Pop, P.; Eles, P.; and Z. Peng, "Scheduling with Optimized Communication for Time-Triggered Embedded Systems," *"Proceedings of the Seventh International Workshop on Hardware/Software Codesign,* March 1999.

77. Bender, M. A.; Muthukrishnan, S.; and R. Rajarama, "Improved Algorithms for Stretch Scheduling," *Proceedings of the Thirteenth Annual ACM-SIAM Symposium on Discrete Algorithms.* January 2002.

78. Nieh, J., and M. S. Lam, "The Design, Implementation and Evaluation of SMART: a Scheduler for Multimedia Applications." *ACM SIGOPS Operating Systems Review, Proceedings of the Sixteenth ACM Symposium on Operating Systems Principles,* October 1997.

79. Dertouzos, M. L., "Control Robotics: The Procedural Control o: Physical Processes," *Proc. IFIP Congress,* 1974, pp. 807-813.

80. Kalyanasundaram, B., and K., Pruhs, "Speed Is As Powerful As Clairvoyance," *Journal of the ACM (JACM),* Vol. 47, No. 4, July, 2002, pp. 617-643.

81. Lam, T, and K. To, "Performance Guarantee for Online Deadline Scheduling in the Presence of Overload," *Proceedings of the TWELFTH ANNUal ACM-SIAM Symposium on Discrete Algorithms,* January 7-9,2001, pp. 755-764.

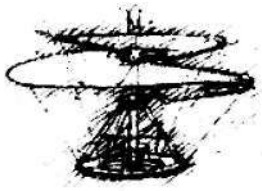
82. Koo, C; Lam, T; Ngan, T; and K. To, "Extra Processors Versus Future Information in Optimal Deadline Scheduling," *Proceedings of the Fourteenth Annual ACM Symposium on Parallel Algorithms and Architectures,* 2002, pp. 133-142.

Real and Virtual Memory

Lead me from the unreal to the real!
—The Upanishads —

Part 3

Memory is second only to processors in importance and in the intensity with which it is managed by the operating system. The next three chapters follow the elegant evolution in memory organizations from the earliest simple single-user, real memory systems to today's popular virtual memory multiprogramming systems. You will learn the motivation for virtual memory and you will focus on schemes for implementing it—paging, segmentation and a combination of the two. You will study the three key types of memory management strategies: fetch (both demand and anticipatory), placement and replacement. You will see that a crucial factor for performance in virtual memory systems is an effective page replacement strategy when available memory becomes scarce, and you will learn a variety of these strategies and Denning's working set model.



The fancy is indeed no other than a mode of memory emancipated from the order of time and space.

—Samuel Taylor Coleridge—

Nothing ever becomes real till it is experienced—even a proverb is no proverb to yon till your life has illustrated it

—John Keats—

Let him in whose ears the low-voiced
Best is killed by the clash of the First,
Who holds that if way to the
Better there be, it exacts a full look at the worst, ...

—Thomas Hardy—

Remove not the landmark on the boundary of the fields.

—Amenemope—

Protection is not a principle, but an expedient.

—Benjamin Disraeli—

A great memory does not make a philosopher, any more than a dictionary can be called a grammar.

—John Henry Cardinal Newman—

Chapter 9

Real Memory Organization and Management

Objectives

After reading this chapter, you should understand:

- *the need for real (also called physical) memory management.*
- *the memory hierarchy.*
- *contiguous and noncontiguous memory allocation.*
- *fixed- and variable-partition multiprogramming.*
- *memory swapping.*
- *memory piacement strategies.*

Chapter Outline

9.1
Introduction

9.2
Memory Organization
Operating Systems Thinking: *There Are No upper limits to Processing Power, Memory, Storage and Bandwidth*

9.3
Memory Management

9.4
Memory Hierarchy

9.5
Memory Management Strategies

9.6
Contiguous vs. Noncontiguous Memory Allocation

9.7
Single-User Contiguous Memory Allocation

9.7.1 *Overlays*
9.7.2 *Protection in a Single-User System*
9.7.3 *Single-Stream Batch Processing*
Operating Systems Thinking: *Change Is the Rule Rather Than the Exception*

9.8
Fixed-Partition Multiprogramming
Anecdote: *Compartmentalization*
Operating Systems Thinking: *Spatial Resources and Fragmentation*

9.9
Variable-Partition Multiprogramming
9. 9.1 Variable-Partition Characteristics
9. 9.2 Memory Placement Strategies

9.10
Multiprogramming with Memory Swapping

Web Resources | Summary | Key Terms | Exercises | Recommended Reading | Works Cited

9.1 Introduction

The organization and management of the real memory (also called main memory, physical memory or primary memory) of a computer system has been a major influence on operating systems design.[1] Secondary storage—most commonly disk and tape - provides massive, inexpensive capacity for the abundance of programs and data that must be kept readily available for processing. It is, however, slow and not directly accessible to processors. To be run or referenced directly, programs and data must be in main memory.

In this and the next two chapters, we discuss many popular schemes for organizing and managing a computer's memory. This chapter deals with real memory; Chapters 10 and 11 discuss virtual memory. We present the schemes approximately as they evolved historically. Most of today's systems are virtual memory systems, so this chapter is primarily of historical value. However, even in virtual memory systems, the operating system must manage real memory. Further, some systems, such as certain types of real-time and embedded systems, cannot afford the overhead of virtual memory—so to them, real memory management remains crucial. Many of the concepts presented in this chapter lay the groundwork for the discussion of virtual memory in the next two chapters.

9.2 Memory Organization

Historically, main memory has been viewed as a relatively expensive resource. As such, systems designers have attempted to optimize its use. Although its cost has declined phenomenally over the decades (roughly according to Moore's Law, as discussed in Chapter 2; see the Operating Systems Thinking feature, There Are No Upper Limits to Processing Power, Memory, Storage and Bandwidth), main memory is still relatively expensive compared to secondary storage. Also, today's operating systems and applications require ever more substantial quantities (Fig. 9.1). For example, Microsoft recommends 256MB of main memory to efficiently run Windows XP Professional.

We (as operating system designers) view main memory in terms of memory organization. Do we place only a single process in main memory, or do we place several processes in memory at once (i.e., do we implement multiprogramming)? If main memory contains several processes simultaneously, do we give each the same amount of space, or do we divide main memory into portions (called partitions) of different sizes? Do define partitions rigidly for extended periods, or dynamically, allowing the system to adapt quickly to changes in the needs of processes? Do we require that processes run in a specific partition, or anywhere they will fit? Do we require the system to place each process in one contiguous block of memory locations, or allow it to divide processes into separate blocks and place them in any available slots in main memory? Systems have been based on each of these schemes. This chapter discusses how each scheme is implemented.

Self Review

1. Why is it generally inefficient to allow only one process to be in memory at a time?
2. What would happen if a system allowed many processes to be placed in main memory, but did not divide memory into partitions?

Ans: **1)** If the single process blocks for I/O, no other processes can use the processor **2)** The processes would share all their memory. Any malfunctioning or malicious process could damage any or all of the other processes.

9.3 Memort Management

Regardless of which memory organization scheme we adopt for a particular system, we must decide which strategies to use to obtain optimal memory performance.[2] **Memory management strategies** determine how a particular memory organization performs under various loads. Memory management is typically performed by both software and special-purpose hardware.

Operating Systems Thinking

There Are No Upper limits to Processing Power, Memory, Storage and Bandwith

Computing is indeed a dynamic field. Processors keep getting faster (and cheaper per executed instruction), main memories keep getting larger (and cheaper per byte), secondary storage media keep getting larger (and cheaper per bit), and communications bandwidths keep getting wider (and cheaper per bit transferred)—all kinds of new devices are being created to interface with, or be integrated into, computers.

Operating systems designers must stay apprised of these trends and the relative rates at which they progress; these trends have an enormous impact on what capabilities operating systems need to have. Early operating systems did not provide capabilities to support computer graphics, graphical user interfaces, networking, distributed computing, Web services, multiprocessing, multithreading, massive parallelism, massive virtual memories, database systems, multimedia, accessibility for people with disabilities, sophisticated security capabilities and so on. All of these innovations over the last few decades have had a profound impact on the requirements for building contemporary operating systems, as you will see when you read the detailed case studies on the Linux and Windows XP operating systems in Chapters 20 and 21. All of these capabilities have been made possible by improving processing power, storage and bandwidth. These improvements will continue, eventually enabling even more sophisticated applications and the operating systems capabilities to support them.

Operating System	*Release Date*	*Minimum Memory Requirement*	*Recommended Memory*
Windows 1.0	November 1985	256KB	
Windows 2.03	November 1987	320KB	
Windows 3.0	March 1990	896KB	1MB
Winndows 3.1	April 1992	2.6MB	4MB
Windows 95	August 1995	8MB	16MB
Windows NT 4.0	August 1996	32MB	96MB
Windows 98	June 1998	24MB	64MB
Windows ME	September 2000	32MB	128MB
Windows 2000 Professional	February 2000	64MB	128MB
Windows XP Home	October 2001	64MB	128MB
Windows XP Professional	October 2001	128MB	256MB

Figure 9.1 | *Microsoft Windows operating system memory requirements.*[3, 4, 5]

The **memory manager** is an operating system component concerned with the system's memory organization scheme and memory management strategies. The memory manager determines how available memory space is allocated to processes and how to respond to changes in a process's memory usage. It also interacts with special-purpose memory management hardware (if any is available) to improve performance. In this and the next two chapters, we describe several different memory management and organization strategies.

Each memory management strategy differs in how it answers certain questions. When does the strategy retrieve a new program and its data to place in memory? Does the strategy retrieve the program and its data when the system specifically asks for it, or does the strategy attempt to anticipate the system's requests? Where in main memory does the strategy place the next program to be run and that program's data? Does it minimize wasted space by packing programs and data as tightly as possible into available memory areas, or does it minimize execution time, placing programs and data as quickly as possible?

If a new program or new data must be placed in main memory and if main memory is currently full, which programs or data already in memory does the strategy replace? Should it replace those that are oldest, those that are used least frequently, or those that were used least recently? Systems have been implemented using these and other memory management strategies.

Self Review

1. When is it appropriate for a memory manager to minimize wasted memory space?
2. Why should memory management organizations and strategies be as transparent as possible to processes?

Ans: **1)** When memory is more expensive than the processor-time overhead incurred by placing programs as tightly as possible into main memory. Also, when the system needs to keep the largest possible contiguous memory region available for large incoming programs and data. **2)** Memory management transparency improves application portability and facilitates development, because the programmer is not concerned with memory management strategies. It also allows memory management strategies to be changed without rewriting applications.

9.4 Memory Hierarchy

In the 1950s and 1960s, main memory was extremely expensive—as much as one dollar per bit! To put that in perspective, Windows XP Professional's recommended 256MB of memory would have cost over 2 billion dollars! Designers made careful decisions about how much main memory to place in a computer system. An installation could buy no more than it could afford but had to buy enough to support the operating system and a given number of processes. The goal was to buy the minimum amount that could adequately support the anticipated workloads within the economic constraints of the installation.

Programs and data must be in main memory before the system can execute or reference them. Those that the system does not need immediately may be kept in secondary storage until needed, then brought into main memory for execution or refer ence. Secondary storage media, such as tape or disk, are generally far less costly per bit than main memory and have much greater capacity. However, main memory may generally be accessed much faster than secondary storage—in today's systems, disk data transfer may be six orders of magnitude slower than that of main memory.[6, 7]

The memory hierarchy contains levels characterized by the speed and cost of memory in each level. Systems move programs and data back and forth between the various levels.[8, 9] This shuttling can consume system resources, such as processor time, that could otherwise be put to productive use. To increase efficiency, current systems include hardware units called memory controllers that perform memory transfer operations with virtually no computational overhead. As a result, systems that exploit the memory hierarchy benefit from lower costs and enlarged capacity.

In the 1960s, it became clear that the memory hierarchy could achieve dramatic improvements in performance and utilization by adding one higher level.[10, 11] This additional level, called cache, is much faster than main memory and is typically located on each processor in today's systems.[12, 13] A processor may reference programs and data directly from its cache. Cache memory is extremely expensive compared to main memory, and therefore only relatively small caches are used. Figure 9.2 shows the relationship between cache, primary memory and secondary storage.

Cache memory imposes one more level of data transfer on the system. Programs in main memory are transferred to the cache before being executed—executing programs from cache is much faster than from main memory. Because many processes that access data and instructions once are likely to access them again in the future (a phenomenon known as temporal locality), even a relatively small

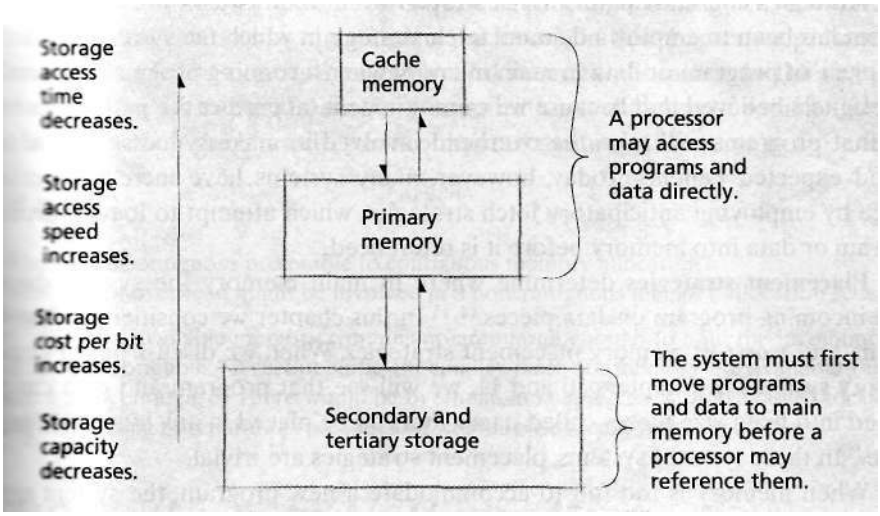


Figure 9.2 | *Hierarchical memory organization*

cache can significantly increase performance (when compared to running programs in a system without cache). Some systems use several levels of cache.

Self Review

1. (T/F) The low cost of main memory coupled with the increase in memory capacity in most systems has obviated the need for memory management strategies**.**

2. How does a program executing a loop benefit from cache memory?

Ans: 1) False. Despite the low cost and high capacity of main memory, there continue to be environments that consume all available memory. Also, memory management strategies should be applied to cache, which consists of more expensive, low-capacity memory. In either case, when memory becomes full, a system must implement memory management strategies to obtain the best possible use of memory. 2) A program executing a loop repeatedly executes the same set of instructions and may also reference the same data. If these instructions and data fit in the cache, the processor can access those instructions and data more quickly from cache than from main memory, leading to increased performance.

9.5 Memory Management Strategies

Memory management strategies are designed to obtain the best possible use of main memory. They are divided into:

1. Fetch strategies
2. Placement strategies
3. Replacement strategies

Fetch strategies determine when to move the next piece of a program or data to main memory from secondary storage. We divide them into two types—demand

fetch strategies and **anticipatory fetch strategies.** For many years, the conventional wisdom has been to employ a demand fetch strategy, in which the system places the next piece of program or data in main memory when a running program references it. Designers believed that because we cannot in general predict the paths of execution that programs will take, the overhead involved in making guesses would far exceed expected benefits. Today, however, many systems have increased performance by employing anticipatory fetch strategies, which attempt to load a piece of program or data into memory before it is referenced.

Placement strategies determine where in main memory the system should place incoming program or data pieces.[14]-[15] In this chapter we consider the **first-fit**, **best-fit,** and **worst-fit** memory placement strategies. When we discuss paged virtual memory systems in Chapters 10 and 11, we will see that program and data can be divided into fixed-size pieces called pages that can be placed in any available "page frame." In these types of systems, placement strategies are trivial.

When memory is too full to accommodate a new program, the system must remove some (or all) of a program or data that currently resides in memory. The system's **replacement strategy** determines which piece to remove.

Self Review

1. Is high resource utilization or low overhead more important to a placement strategy?
2. Name the two types of fetch strategies and describe when each one might be more appropriate than the other.

Ans: **1)** The answer depends on system objectives and the relative costs of resources and overhead. In general, the operating system designer must balance overhead with high memory utilization to meet the system's goals. **2)** The two types are demand fetch and anticipator fetch. If the system cannot predict future memory usage with accuracy, then the lower overhead of demand fetching results in higher performance and utilization (because the system does not load from disk information that will not be referenced). However, if programs exhibit predict able behavior, anticipatory fetch strategies can improve performance by ensuring that pieces of programs or data are located in memory before processes reference them.

9.6 Contiguous vs. Noncontigues Memory Allocation

To execute a program in early computer systems, the system operator or the operating system had to find enough contiguous main memory to accommodate the entire program. If the program was larger than the available memory, the system could not execute it. In this chapter, we discuss the early use of this method, known as **contiguous memory allocation,** and some problems it entailed. When researchers attempted to solve these problems, it became clear that systems might benefit from noncontiguous memory allocation.[16]

In **noncontiguous memory allocation,** a program is divided into blocks or **segments** that the system may place in nonadjacent slots in main memory. This allows making use of holes (unused gaps) in memory that would be too small to hold whole programs. Although the operating system thereby incurs more overhead, this

can be certified by the increase in the level of multiprogramming (i.e., the number of processes that can occupy main memory at once). In this chapter we present the techniques that led to noncontiguous physical memory allocation. In the next two chapters we discuss the virtual memory organization techniques of paging and segmentation, each of which requires noncontiguous memory allocation.

Self Review

1. When is noncontiguous preferable to contiguous memory allocation?
2. What sort of overhead might be involved in a noncontiguous memory allocation scheme?

Ans: 1) When available memory contains no area large enough to hold the incoming program is one contiguous piece, but sufficient smaller pieces of memory are available that, in total, are large enough. 2) There would be overhead in keeping track of available blocks and blocks that belong to separate processes, and where those blocks reside in memory.

9.7 Single-User Contiguous Memory Allocation

Early computer systems allowed only one person at a time to use a machine. All the machine's resources were dedicated to that user. Billing was straightforward—the user was charged for all the resources whether or not the user's job required them. In fact, the normal billing mechanisms were based on **wall clock time.** The system operator gave the user the machine for some time interval and charged a flat hourly rate.

Figure 9.3 illustrates the memory organization for a typical **single-user contiguous memory allocation system.** Originally, there were no operating systems—the programmer wrote all the code necessary to implement a particular application, including the highly detailed machine-level input/output instructions. Soon, system

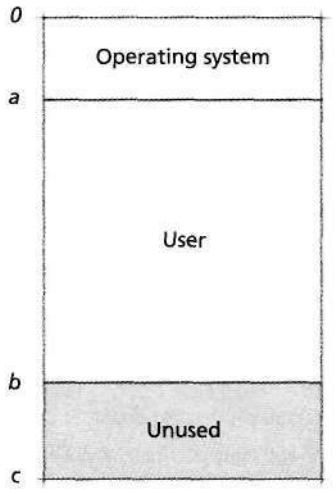


Figure 9.3 | *Single-user contiguous memory allocation.*

designers consolidated input/output coding that implemented basic functions into an input/output control system (IOCS).[17] The programmer called IOCS routines to do the work instead of having to "reinvent the wheel" for each program. The IOCS greatly simplified and expedited the coding process. The implementation of input/output control systems may have been the beginning of today's concept of operating systems.

9.7.1 Overlays

We have discussed how contiguous memory allocation limited the size of programs that could execute on a system. One way in which a software designer could overcome the memory limitation was to create overlays, which allowed the system to execute programs larger than main memory. Figure 9.4 illustrates a typical overlay. The programmer divides the program into logical sections. When the program does not need the memory for one section, the system can replace some or all of it with the memory for a needed section.[18]

Overlays enable programmers to "extend" main memory. However, manual overlay requires careful and time-consuming planning, and the programmer often

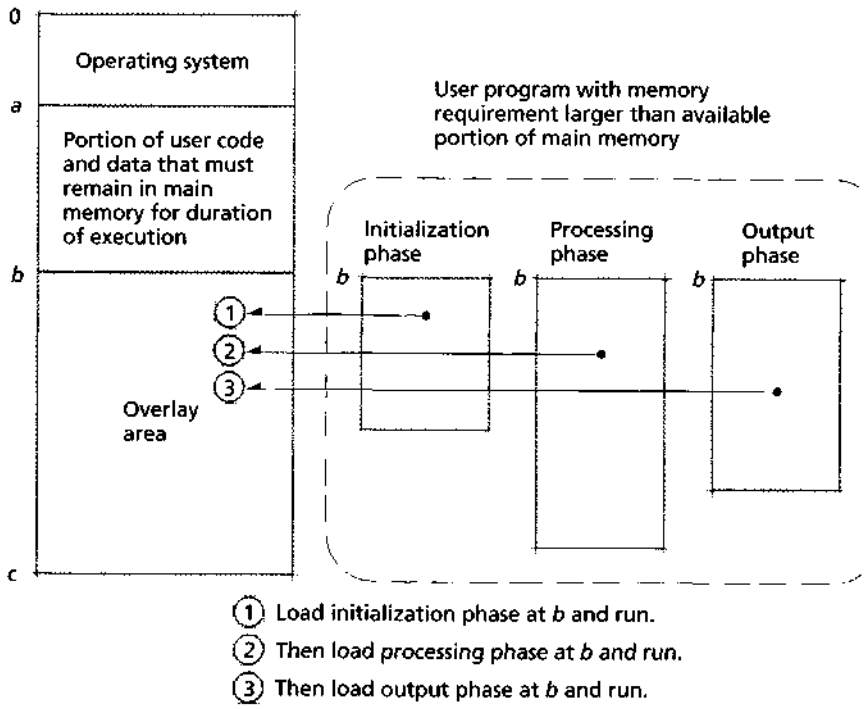


Figure 9.4 | *Overlay structure.*

must have detailed knowledge of the system's memory organization. A program with a sophisticated overlay structure can be difficult to modify. Indeed, as programs grew in complexity, by some estimates as much as 40 percent of programming expenes were for organizing overlays.[19] It became clear that the operating system needed to insulate the programmer from complex memory management tasks such as overlays. As we will see in subsequent chapters, virtual memory systems obviate the need for programmer-controlled overlays, in the same way that the IOCS freed the programmer from repetitive, low-level I/O manipulation.

Self review

1. How did the IOCS facilitate program development**?**
2. Describe the costs and benefits of overlays.

Ans: 1) Programmers were able to perform I/O without themselves having to write the low-level commands that were now incorporated into the IOCS, which all programmers could use instead of having to "reinvent the wheel." 2) Overlays enabled programmers to write programs larger than real memory, but managing these overlays increased program complexity, which increased the size of programs and the cost of software development.

9.7.2 Protection in a Single-User System

In single-user contiguous memory allocation systems, the question of protection is simple. How should the operating system be protected from destruction by the user's program?

A process can interfere with the operating system's memory—either intentionally or inadvertently—by replacing some or all of its memory contents with other data. If it destroys the operating system, then the process cannot proceed. If the process attempts to access memory occupied by the operating system, the user can detect the problem, terminate execution, possibly fix the problem and relaunch the program.

Without protection, the process may alter the operating system in a more subthe, nonfatal manner. For example, suppose the process accidentally changes certain input/output routines, causing the system to truncate all output records. The process could still run, but the results would be corrupted. If the user does not examine the results until the process completes, then the machine resource has been wasted. Worse yet, the damage to the operating system might cause outputs to be produced that the user cannot easily determine to be inaccurate. Clearly, the operating system must be protected from processes.

Protection in single-user contiguous memory allocation systems can be implemented with a single **boundary register** built into the processor, as in Fig. 9.5, and which can be modified only by a privileged instruction. The boundary register contains the memory address at which the user's program begins. Each time a process references a memory address, the system determines if the request is for an address greater than or equal to that stored in the boundary register. If so, the system ser----- the memory request. If not, then the program is trying to access the operating

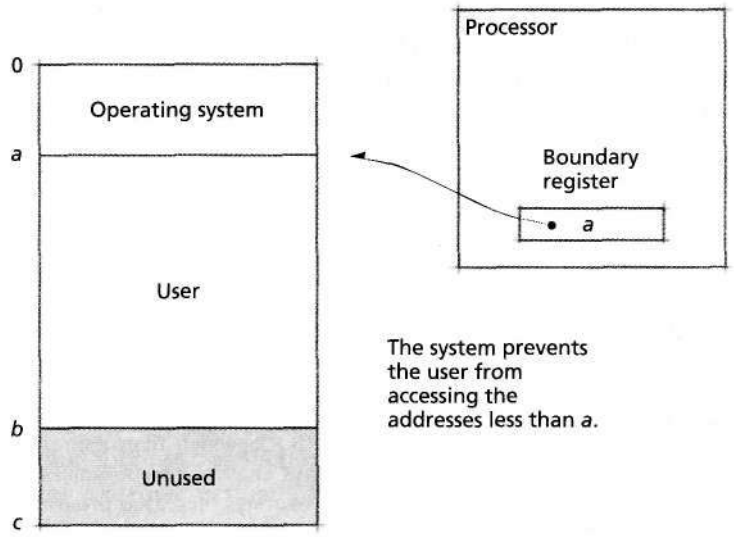


Figure 9.5 | *Memory protection with single-user contiguous memory allocation.*

system. The system intercepts the request and terminates the process with an appropriate error message. The hardware that checks boundary addresses operates quickly to avoid slowing instruction execution.

Of course, the process must access the operating system from time to time to obtain services such as input/output. Several system calls (also called **supervisor calls**) are provided that may be used to request services from the operating system. When a process issues a system call (e.g., to write data to a disk), the system detects the call and switches from the user mode to the kernel mode (or executive mode) of execution. In kernel mode, the processor may execute operating system instructions and access data to perform tasks on behalf of the process. After the system has performed the requested task, it switches back to user mode and returns control to the process.[20]

The single boundary register represents a simple protection mechanism. As operating systems have become more complex, designers have implemented more sophisticated mechanisms to protect the operating system from processes and to protect processes from one another. We discuss these mechanisms in detail later.

Self Review

1. Why is a single boundary register insufficient for protection in a multiuser system?
2. Why are system calls necessary for an operating system?

Ans: 1) The single boundary would protect the operating system from being corruted by user processes, but not protect processes from corrupting each other. **2)** System calls enable processes to request services from the operating system while ensuring that the operating system is protected from its processes.

9.7.3 Single-Stream Batch Processing

Early single-user real memory systems were dedicated to one job for more than the job's execution time. Jobs generally required considerable **setup time** during which the operating system was loaded, tapes and disk packs were mounted, appropriate formsplaced in the printer, time cards "punched in," and so on. When jobs completed, they required considerable **teardown time** as tapes and disk packs were removed, forms removed, time cards "punched out." During job setup and teardown the computer sat idle.

Designers realized that if they could automate various aspects of **job-to-job transition**, they could reduce considerably the amount of time wasted between jobs. This led to the development of **batch-processing** systems (see the Operating Systems Thinking feature, Change Is the Rule Rather Than the Exception). **In single-stream batch processing,** jobs are grouped in batches by loading them consecutively onto tape or disk. A **job stream processor** reads the **job control language** statements (that define each job) and facilitates the setup of the next job. It issues directives to the system operator and performs many functions that the operator previously performed manually. When the current job terminates, the job stream reader reads in the control-language statements for the next job and performs appropriate housekeeping chores to facilitate the transition to the next job. Batch-processing systems greatly improved resource utilization and helped demonstrate the real value of operating systems and **intensive resource management.** Single-stream batch-processing systems were the state of the art in the early 1960s.

Self Review

1. (T/F) Batch-processing systems removed the need for a system operator.
2. What was the key contribution of early batch-processing systems?

Operating Systems Thinking

Change Is the Rule Rather Than the Exception

Historically, change is the rule rather than the exception and that those changes happen faster than anticipated and often are far more wrenching than imagined. You see this in the computer field all the time. Just look at who the leading companies were in the 1960s and 1970s; today many of those companies are gone or have far less prominent positions in the industry. Look at the way computers and operating systems were designed in those decades; today those designs, in many cases, are profoundly different. Throughout the book we discuss many architectural issues and many software engineering issues that designers must consider as they create operating systems that adapt well to change.

Ans: **1)** False. A system operator was needed to set up and "tear down" the jobs and control them as they executed. **2)** They automated various aspects of job-to-job transition, considerably reducing the amount of wasted time between jobs and improved resource utilization.

9.8 Fixed-Partition Multiprogramming

Even with batch-processing operating systems, single-user systems still waste a considerable amount of the computing resource (Fig. 9.6). A typical process would consume the processor time it needed to generate an input/output request; the process could not continue until the I/O finished. Because I/O speeds were extremely slow compared with processor speeds (and still are), the processor was severely underutilized.

Designers saw that they could further increase processor utilization by implementing multiprogramming systems, in which several users simultaneously compete for system resources. The process currently waiting for I/O yields the processor if another process is ready to do calculations. Thus, I/O operations and processor calculations can occur simultaneously. This greatly increases processor utilization and system throughput.

To take maximum advantage of multiprogramming, several processes must reside in the computer's main memory at the same time. Thus, when one process requests input/output, the processor may switch to another process and continue to perform calculations without the delay associated with loading programs from secondary storage. When this new process yields the processor, another may be ready

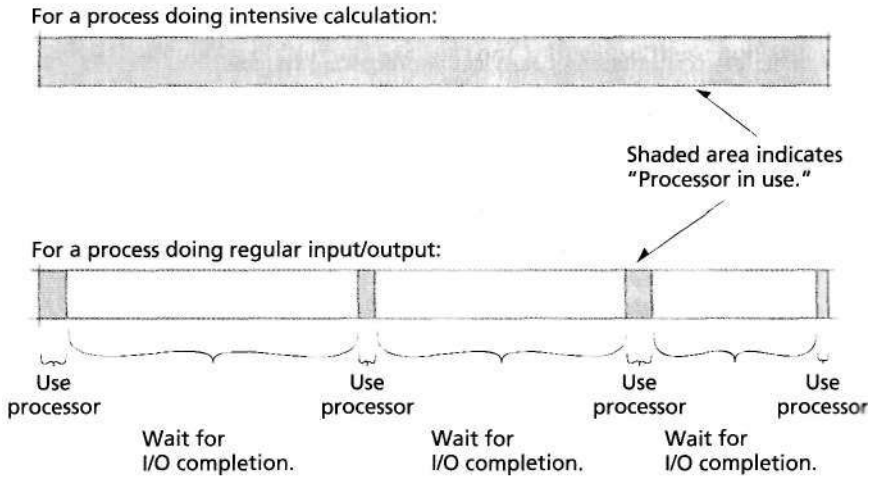


Figure 9.6 *| Procssor utilization on a single-user s y s t e m . [Note: In many single-user jobs, I/O waits are much longer relative to processor utilization periods indicated in this diagram.]*

to use it. Many multiprogramming schemes have been implemented, as discussed in this and the next several sections.

The earliest multiprogramming systems used fixed-partition multiprogramming.[21] Under this scheme, the system divides main memory into a number of fixed-size partitions. Each partition holds a single job, and the system switches the processor rapidly between jobs to create the illusion of simultaneity.[22] This technique enables the system to provide simple multiprogramming capabilities. Clearly, multiprogramming normally requires more memory than does a single-user system. However, the improved resource use for the processor and the peripheral devices justifies the expense of the additional memory.

In the earliest multiprogramming systems, the programmer translated a job using an absolute assembler or compiler (see Section 2.8, Compiling, Linking and loading). While this made the memory management system relatively straightforward to implement, it meant that a job had its precise location in memory determined before it was launched and could run only in a specific partition (Fig. 9.7).This restriction led to wasted memory If a job was ready to run and the program's partition was occupied, then that job had to wait, even if other partitions were available. Figure 9.8 shows an extreme example. All the jobs in the system must run in partition 3 (i.e., the prrograms' instructions all begin at address c). Because this partition currently is in use. all other jobs are forced to wait, even though the system has two other partitions in which the jobs could run (if they had been compiled for these partitions).

To overcome the problem of memory waste, developers created relocating compilers, assemblers and loaders. These tools produce a relocatable program that can run in any available partition that is large enough to hold that program (Fig. 9.9). This scheme eliminates some of the memory waste inherent in multipro-

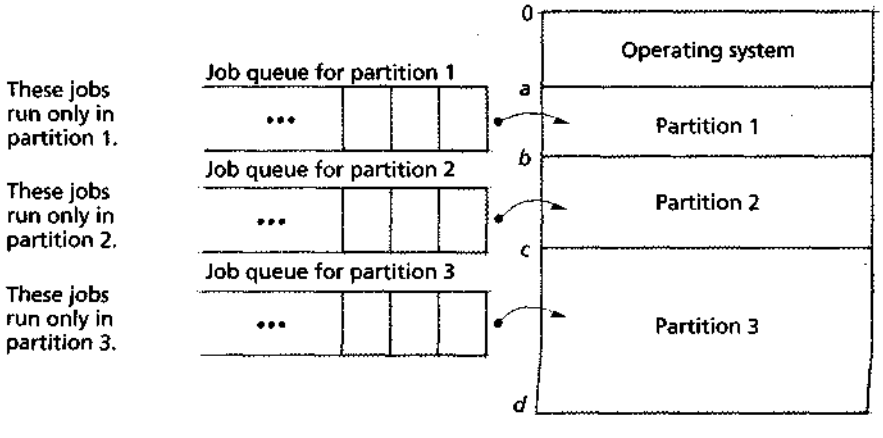


Figure 9.7 *| Fixed-partition multiprogramming with absolute translation and loading.*

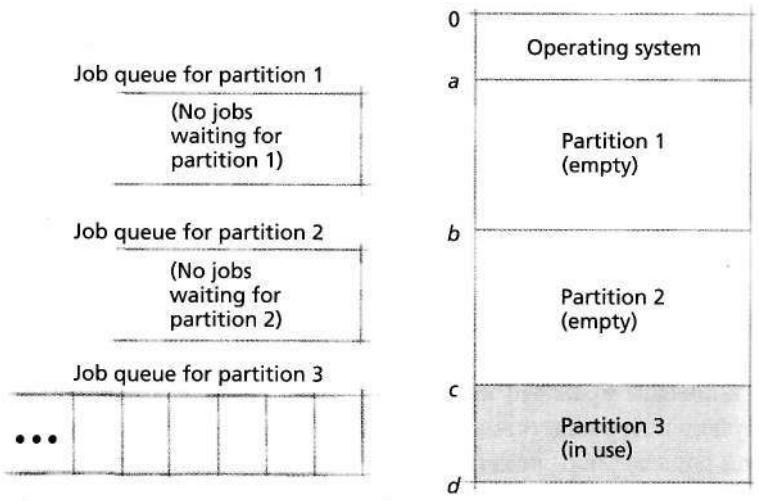


Figure 9.8 | *Memory waste under fixed-partition multiprogramming with absolute translation and loading.*

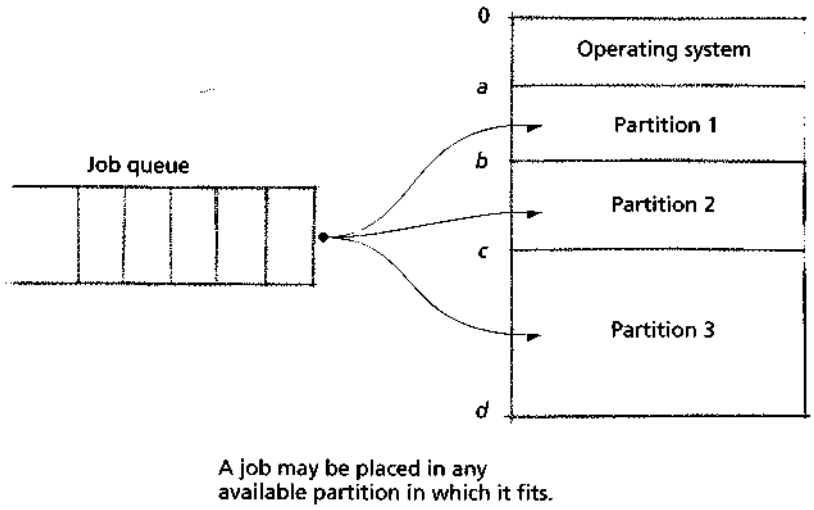


Figure 9.9 | *Fixed-partition multiprogramming with relocatable translation and loading.*

gramming with absolute translation and loading; however, relocating translators and loaders are more complex than their absolute counterparts.

As memory organization increased in complexity, designers had to augment the protection schemes. In a single-user system, the system must protect only the

operating system from the user process. In a multiprogramming system, the system must protect the operating system from all user processes and protect each process from all the others. In contiguous-allocation multiprogramming systems, such as those discussed in this section, protection often is implemented with multiple boundary registers. The system can delimit each partition with two boundary registers-- low and high, also called the **base** and **limit** registers (Fig. 9.10). When a pro-

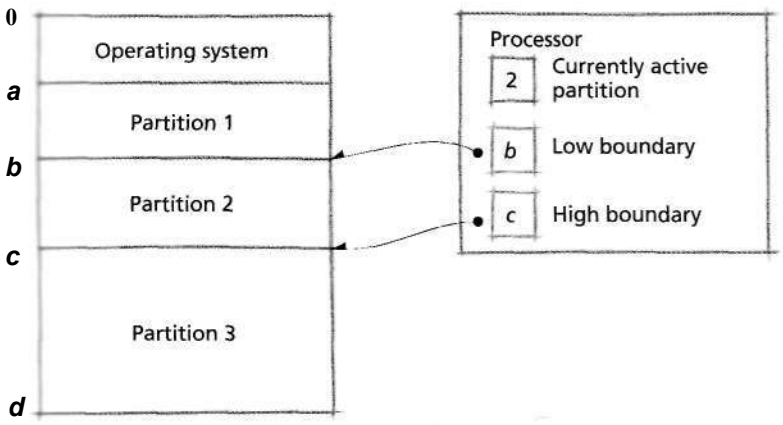


Figure 9.10 | *Memory protection in contiguous-allocation multiprogramming systems.*

Anecdote

Compartmentalization

When HMD was interviewing for a job in computer security at the Pentagon, his last interview of the day was with the person ultimately responsible for all Pentagon computer systems. When offered the job, HMD was enthusiastic: "Yes sir, if I come to the Pentagon you can count on me to work day and night to ensure that the enemy can't compromise our computer systems." The employer smiled and said, "Harvey, I'm not that concerned about the enemy, 1 just want to make sure that the Navy doesn't know what the Air Force is doing."

Lesson to operating systems designers: A key goal of operating systems is compartmentalization. Operating systems must provide a working environment in which a great diversity of users can operate concurrently, knowing that their work is private and protected from other users.

cess issues a memory request, the system checks whether the requested address is greater than or equal to the process's low boundary register value and less than the process's high boundary register value (see the Anecdote, Compartmentalization). If so, the system honors the request; otherwise, the system terminates the program with an error message. As with single-user systems, multiprogramming systems provide system calls that enable user programs to access operating system services.

One problem prevalent in all memory organizations is that of **fragmentation** —the phenomenon wherein the system cannot use certain areas of available main memory (see the Operating Systems Thinking feature, Spatial Resources and Fragmentation).[23] Fixed-partition multiprogramming suffers from **internal fragmentation**, which occurs when the size of a process's memory and data is smaller than that of the partition in which the process executes.[24] (We discuss external fragmentation in Section 9.9, Variable-Partition Multiprogramming.)

Figure 9.11 illustrates the problem of internal fragmentation. The system's three user partitions are occupied, but each program is smaller than its corresponding partition. Consequently, the system may have enough main memory space in which to run another program but has no remaining partitions in which to run the program. Thus, some of the system's memory resources are wasted. In the next section we discuss another memory organization scheme that attempts to solve the problem of fixed partitions. We shall see that, although this scheme makes improvements, the system can still suffer from fragmentation.

Operating Systems Thinking

Spatial Resources and Fragmentation

Consider a common scenario: A restaurant in a shopping mall wants to expand, but no larger unoccupied stores are available in the mall, so the mall owners must wait for adjacent stores to become available, and then knock down some walls to create a larger space. An office building may initially start off as en empty "shell;" as customers rent spaces, the walls are built to delineate the separate office spaces. When a customer wants to expand its space, it may be difficult to do so because no larger space may be available and the adjacent spaces may not be available. In both of these cases the units desiring to expand may not be able to find a sufficiently large contiguous space and may have to settle for using several noncontiguous, smaller spaces. This is called fragmentation and it is a recurrirg theme in operating systems.In this book, you will see how main memory and secondary storage can each suffer various forms of fragmentation and how operat ing systems designers deal with these problems.

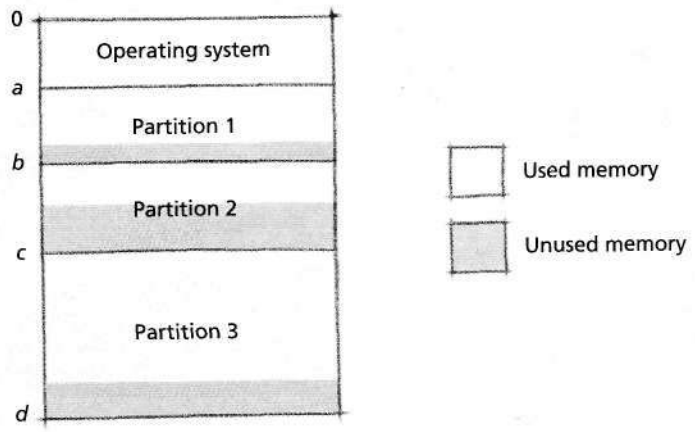


Figure 9.11 | *Internal fragmentation in a fixed-partition multiprogramming system.*

Self Review

1. Explain the need for relocating compilers, assemblers and loaders.
2. Describe the benefits and drawbacks of large and small partition sizes.

Ans: 1) Before such tools, programmers manually specified the partition into which their program had to be loaded, which potentially wasted memory and processor utilization, and reduced application portability. 2) Larger partitions allow large programs to run, but result in internal fragmentation for small programs. Small partitions reduce the amount of internal fragmentation and increase the level of multiprogramming by allowing more programs to reside in memory at once, but limit program size.

9.9 Variable-Partition Multiprogramming

Fixed-partition multiprogramming imposes restrictions on a system that result in inefficient resource use. For example, a partition may be too small to accommodate a waiting process, or so large that the system loses considerable resources to internal fragmentation. An obvious improvement, operating system designers decided, would be to allow a process to occupy only as much space as needed (up to the amount of available main memory). This scheme is called **variable-partition multiprogramming**.[25 26 27]

9.9.1 Variable-Partition Characteristics

Figure 9.12 shows how a system allocates memory under variable-partition multiprogramming. We continue to discuss only contiguous-allocation schemes, where a process must occupy adjacent memory locations. The queue at the top of the figure contains available jobs and information about their memory requirements. The operating system makes no assumption about the size of a job (except that it does

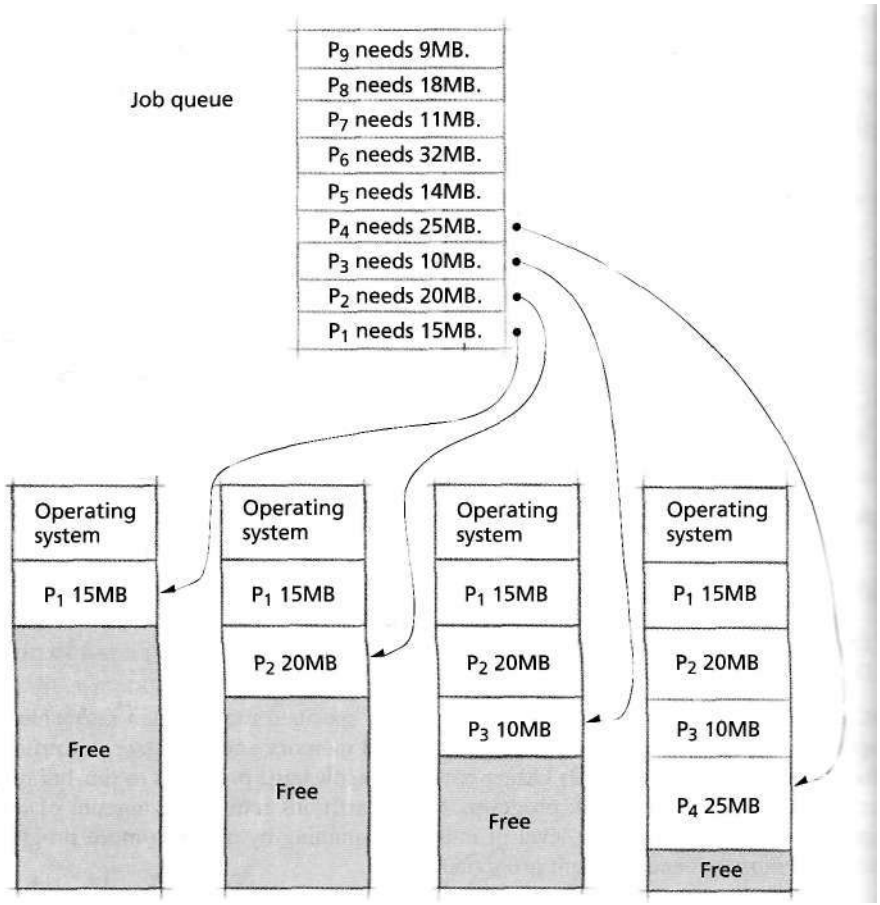


Figure 9.12 | *Initial partition assignments in variable partition programming*

not exceed the size of available main memory). The system progresses through the queue and places each job in memory, where there is available space, at which point it becomes a process. In Fig. 9.12, main memory can accommodate the first four jobs; we assume the free space that remains after the system has placed the job corresponding to process P_4 is less than 14KB (the size of the next available job).

Variable-partition multiprogramming organizations do not suffer from internal fragmentation, because a process's partition is exactly the size of the process. But every memory organization scheme involves some degree of waste. In variable partition multiprogramming, the waste does not become obvious until processes finish and leave **holes** in main memory, as shown in Fig. 9.13. The system can continue to place new processes in these holes. However, as processes continue to complete, the holes get smaller, until every hole eventually becomes too small to hold a new process. This is called **external fragmentation,** where the sum of the holes is

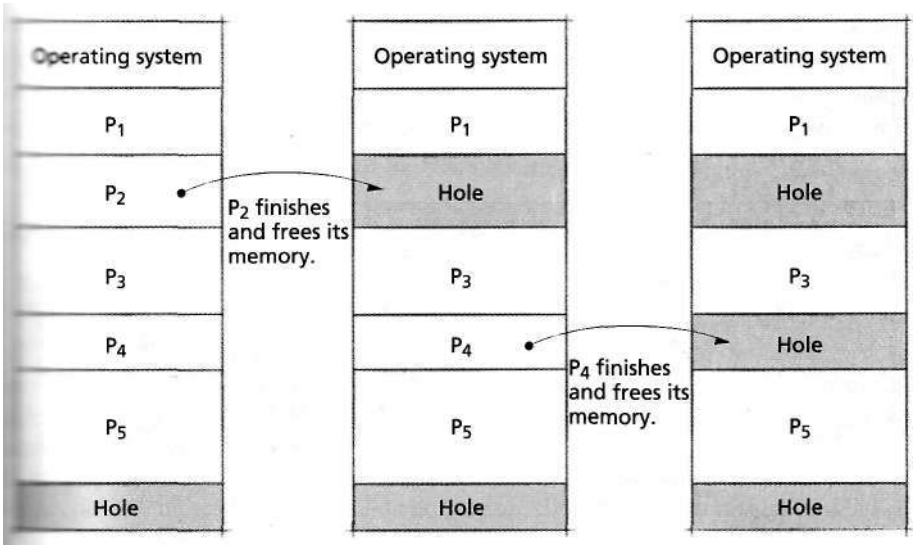


Figure 9.13 | *Memory "holes" in variable-partition multiprogramming.*

enough to accommodate another process, but the size of each hole is too small to accomodate any available process.[28]

The system can take measures to reduce some of its external fragmentation. When a process in a variable-partition multiprogramming system terminates, the system can determine whether the newly freed memory area is adjacent to other free memory areas. The system then records in a **free memory list** either (1) that the system now has an additional hole or (2) that an existing hole has been enlarged (reflecting the merging of the existing hole and the new adjacent hole).[29]-[30] The pro- (reflecting the merging adjacent holes to form a single, larger hole is called **coalescing** and is illustrated in Fig. 9.14. By coalescing holes, the system reclaims the largest possible contiguous blocks of memory.

Even as the operating system coalesces holes, the separate holes distributed throughout main memory rnay still constitute a significant amount of memory—enough in total to satisfy a process's memory requirements, although no individual hole is large enough to hold the process.

Another technique for reducing external fragmentation is called **memory compaction** (Fig. 9.15), which relocates all occupied areas of memory to one end or the other of main memory.[31] This leaves a single large free memory hole instead of the numerous small holes common in variable-partition multiprogramming. Now all of the available free memory is contiguous, so that an available process can run if its memory requirement is met by the single hole that results from compaction.

Sometimes memory compaction is colorfully referred to as **burping the memory.** More conventionally, it is called **garbage collection.**[32]

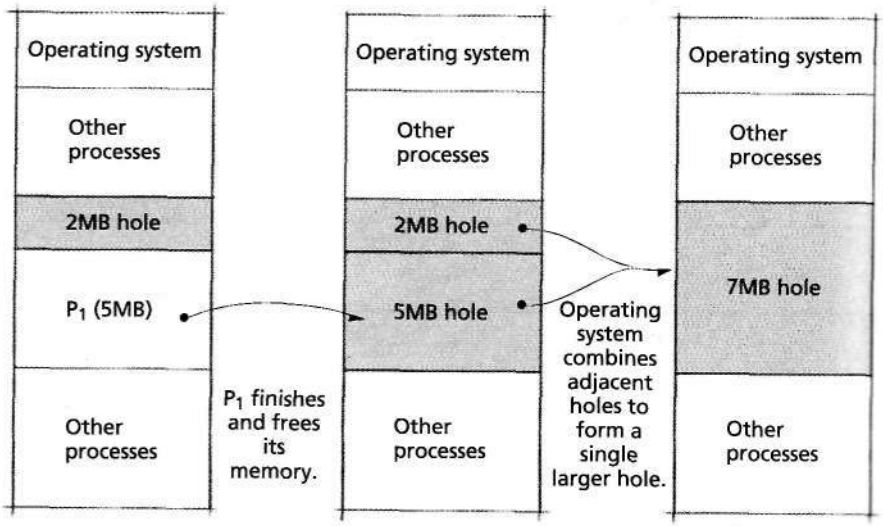


Figure 9.14 | *Coalescing memory "holes" in variable partition multiprogramming*

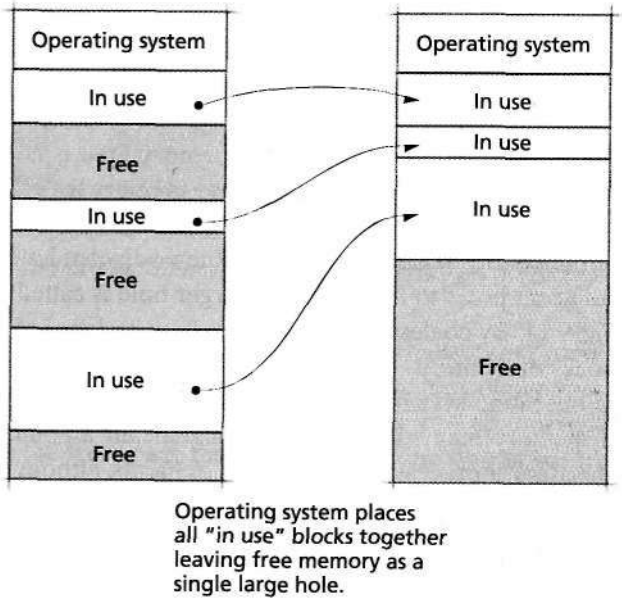


Figure 9.15 | *Memory compaction in variable-partition multiprogramming.*

Compaction is not without drawbacks. Compaction overhead consumes system resources that could otherwise be used productively. The system also must cease all other computation during compaction. This can result in erratic response

times for interactive users and could be devastating in real-time systems. Furthermore, compaction must relocate the processes that currently occupy the main memory. This means that the system must now maintain relocation information, which ordinary is lost when the system loads a program. With a normal, rapidly changing job mix, the system may compact frequently. The consumed system resources might not justify the benefits of compaction.

Self Review

1. Explain the difference between internal fragmentation and external fragmentation**,**
2. Describe two techniques to reduce external fragmentation in variable-partition multiprogramming systems.

Ans: 1) Internal fragmentation occurs in fixed-partition environments when a process is allocated more space than it needs, leading to wasted memory space inside each partition. External fragmentation occurs in variable-partition environments when memory is wasted due to holes developing in memory between partitions. **2)** Coalescing merges adjacent free memory blocks into one larger block. Memory compaction relocates partitions to be adjacent to one another to consolidate free memory into a single block.

9.9.2 Memory Placement Strategies

In a variable-partition multiprogramming system, the system often has a choice as to which memory hole to allocate for an incoming process. The system's **memory placement strategy** determines where in main memory to place incoming programs and data.[33, 34, 35] Three strategies frequently discussed in the literature are illustrated in Fig. 9.16.[36]

- **First-fit strategy**—The system places an incoming job in main memory in the first available hole that is large enough to hold it. First-fit has intuitive appeal in that it allows the system to make a placement decision quickly.
- **Best-fit strategy**—The system places an incoming job in the hole in main memory in which it fits most tightly and which leaves the smallest amount of unused space. To many people, best fit is the most intuitive strategy, but it requires the overhead of searching all of the holes in memory for the best fit and tends to leave many small, unusable holes. Note in Fig. 9.16 that we maintain the entries of the free memory list in ascending order; such sorting is relatively expensive.
- **Worst-fit strategy**—At first this appears to be a whimsical choice. Upon closer examination, though, worst-fit also has strong intuitive appeal. Worst fit says to place a job in main memory in the hole in which it fits worst (i.e., in the largest possible hole). The intuitive appeal is simple: After the job is placed in this large hole, the remaining hole often is also large and thus able to hold a relatively large new program. The worst-fit strategy also requires the overhead of finding the largest hole and tends to leave many small, unusable holes.

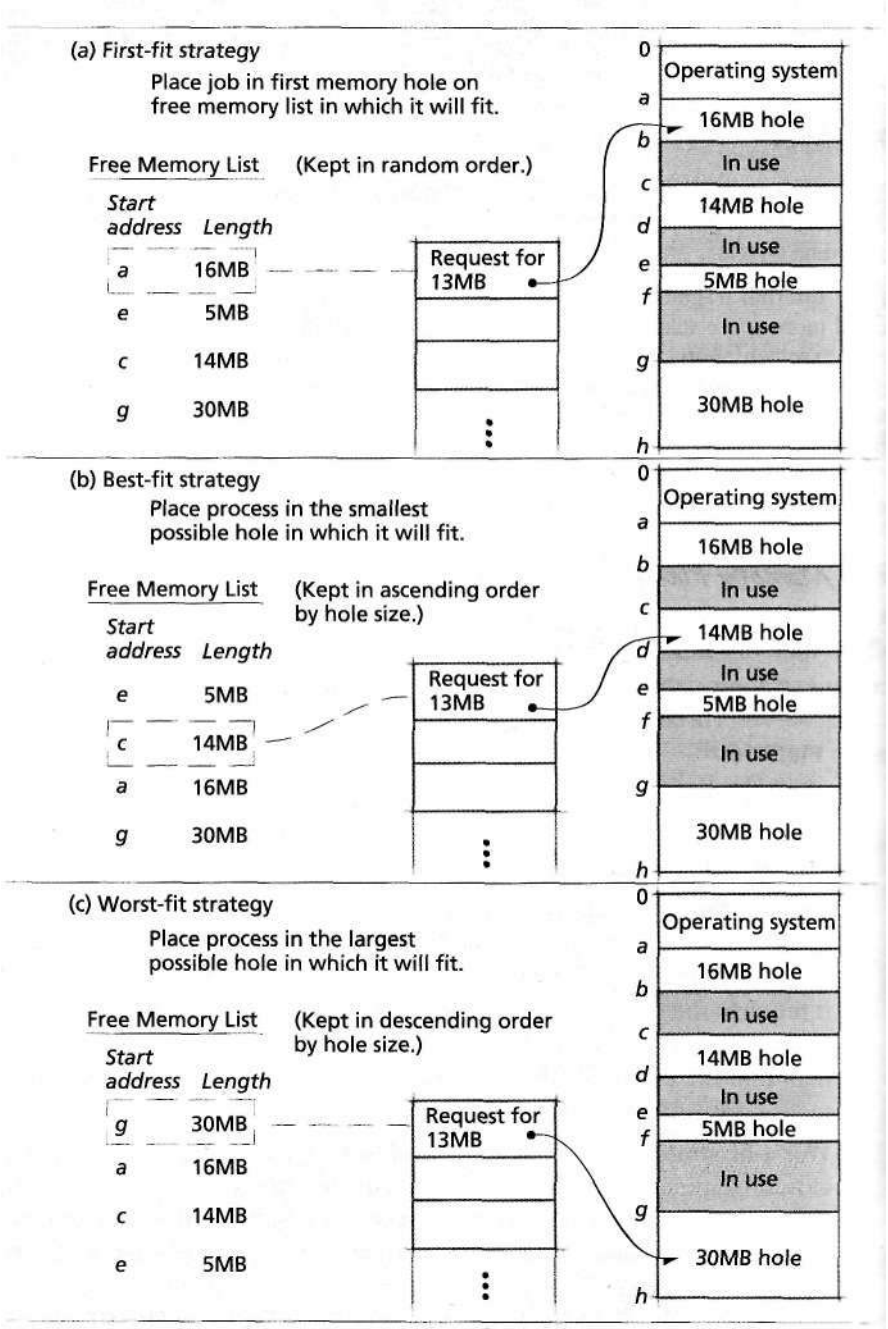


Figure 9.16 | *First-fit, best-fit and worst-fit memory placement strategies.*

A variation of first-fit, called the next-fit strategy, begins each search for an available hole at the point where the previous search ended.[37] Exercise 9.20 at the end of this chapter examines the next-fit strategy in detail.

Self Review

1. Why is first-fit an appealing strategy?
2. (T/F) None of the memory placement strategies in this section result in internal fragmen-

Ans: 1) First-fit is intuitively appealing because it does not require that the free memory list be sorted, so it incurs little overhead. However, it may operate slowly if the holes that are too small to hold the incoming job are at the front of the free memory list. **2)** True.

9.10 Multiprogramming with Memory Swapping

In all the multiprogramming schemes we have discussed in this chapter, the system maintains a process in main memory until it completes. An alternative to this scheme is **swapping**, in which a process does not necessarily remain in main memory throughout its execution.

In some swapping systems (Fig. 9.17), only one process occupies main memory at a given time. That process runs until it can no longer continue (e.g., because it must wait for I/O completion), at which time it relinquishes both the memory and the processor to the next process. Thus, the system dedicates the entire memory to one process for a brief period. When the process relinquishes the resource, the system swaps (or **rools**) out the old process and swaps (or rolls) in the next process. To swap a process out, the system stores the process's memory contents (as well as its PCB) in secondary storage. When the system swaps the process back in, the process's memory contents and other values are retrieved from secondary storage. The system normally swap a process in and out many times before the process completes.

Many early timesharing systems were implemented with this swapping technique. Response times could be guaranteed for a few users, but designers knew that they needed better techniques to handle large numbers of users. The swapping systems of the early 1960s led to today's paged virtual memory systems. Paging is considered in detail in the next two chapters on virtual memory systems.

More sophisticated swapping systems have been developed that allow several processes to remain in main memory at once.[38,39] In these systems, the system swaps a process out only when an incoming process needs that memory space. With a sufficient amount of main memory, these systems greatly reduce the time spent swapping.

Self Review

1. Explain the overhead of swapping in terms of processor utilization. Assume that memory can hold only one process at a time.
2. Why were swapping systems in which only a single process at a time was in main memory insufficient for multiuser interactive systems?

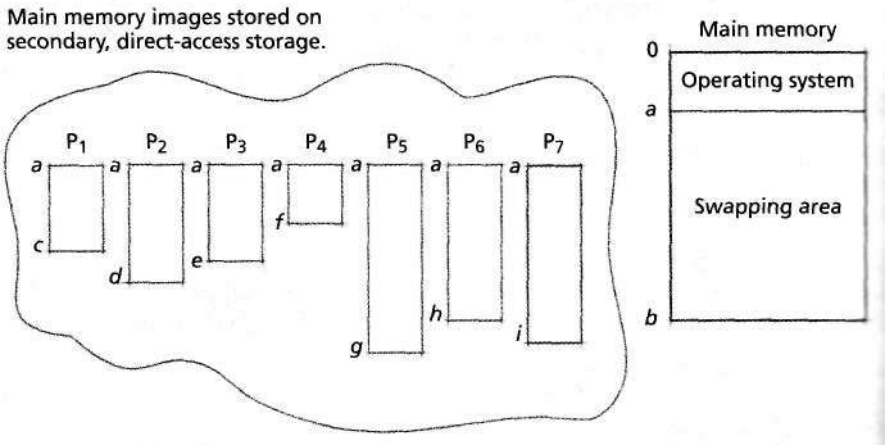


Figure 9.17 | *Multiprogramming in a swapping system in which only a single process at a time is in main memory.*

Ans: 1) Enormous numbers of processor cycles are wasted when swapping a program between disk and memory. **2)** This type of swapping system could not provide reasonable response times to a large number of users, which is required for interactive systems.

Web Resources

www.kingston.com/tools/umg/default.asp
Describes the role of memory in a computer. Although the discussion is geared toward today's computers (which use virtual memory, a topic introduced in the next chapter), it describes many fundamental memory concepts.

www.1inux-mag.com/2001-06/compile_01.html
Describes the working of the mallocO function in C, which allocates memory to newly created variables. The article also considers some basic allocation strategies that were discussed above (including first-fit, next-fit and worst-fit).

www.memorymanagement.org
Presents memory management techniques and garbage collection. The site also contains a glossary of over 400 memory management terms and a beginner's guide to memory management.

Sunmary

The organization and management of the real memory (also called main memory, physical memory or primary memory) of a computer system has been one of the most important influ---es upon operating systems design. Although most of today's systems implement virtual memory, certain types of real-time and embedded systems cannot afford the overhead of virtual memory—real memory management remains crucial to such systems.

Regardless of which memory organization scheme we adopt for a particular system, we must decide which strategies to use to obtain optimal memory performance. Memory management strategies determine how a particular memory organization performs under various policies. Memory management is typically performed by both software and special-purpose hardware. The memory manager is a component of the operating system that determines how available memory space is allocated to processes and how to respond to changes in a process's memory usage. The memory manager also interacts with special-purpose memory management hardware (if any is available) to improve performance.

Programs and data must be in main memory before the system can execute or reference them. The memory hierarchy contains levels characterized by the speed and cost of memory in each level. Systems with several levels of memory perform transfers that move programs and data back and forth between the various levels. Above the main memory level in the hierarchy is cache, which is much faster than main memory and is typically located on each processor in today's systems. Programs in main memory are transferred to the cache, where they are executed much faster than they would be from main memory. Because many processes that access data and instructions once are likely to access them again in the future (a phenomenon known as temporal locality), even a relatively small cache can significantly increase performance (when compared to running programs in a system without cache).

Memory management strategies are divided into fetch strategies, which determine when to move the next piece of a program or data to main memory from secondary storage, placement strategies, which determine where in main memory the system should place incoming program or data pieces, and replacement strategies, which determine which piece of a program or data to replace to accommodate incoming program and data pieces.

Contiguous memory allocation systems store a program in contiguous memory locations. In noncontiguous memory allocation, a program is divided into blocks or segments that the system may place in nonadjacent slots in main memory. This allows the memory management system to make use of holes (unused gaps in memory) that would otherwise be too small to hold programs. Although the operating system incurs more overhead in managing noncontiguous memory allocation, this can be justified by the increase in the level of multiprogramming (i.e., the number of processes that can occupy main memory at once).

Early computer systems allowed only one person at a time to use a machine. These computer systems typically did not contain an operating system. Later, system designers consolidated input/output coding that implemented basic functions into an input/output control system (IOCS), so the programmer no longer had to code input/output instructions directly. With overlays, the system could execute programs larger than main memory. However, manual overlay requires careful and time-consuming planning, and the programmer often must have detailed knowledge of the system's memory organization.

Without protection in a single-user system, a process can interfere with the operating system's memory—either intentionally or inadvertently—by replacing some or all of its memory contents with other data. Protection in single-user contiguous memory allocation systems can be implemented with a single boundary register built into the processor. Processes must access the operating system from time to time to obtain services such as input/output. The operating system provides several system calls (also called supervisor calls) that may be used to request such services from the operating system.

Early single-user real memory systems were dedicated to one job for more than the job's execution time. Jobs generally required considerable setup and teardown time, during which the computer sat idle. The development of batch-processing systems improved utilization. In single-stream batch-processing, jobs are grouped in batches by loading them consecutively onto tape or disk. Even with batch-processing operating systems, single-user systems still wasted a considerable amount of the computing resource. Therefore, designers chose to implement multiprogramming systems, in which several users simultaneously compete for system resources.

The earliest multiprogramming systems used fixed-partition multiprogramming, in which the system divides main memory into a number of fixed-size partitions., each holding a single job. The system switches the processor rapidly between

jobs to create the illusion of simultaneity. To overcome the problem of memory waste, developers created relocating compilers, assemblers and loaders.

In contiguous-allocation multiprogramming systems, protection often is implemented with multiple boundary registers, called the base and limit registers, for each process. One problem prevalent in all memory organizations is that of fragmentation—the phenomenon wherein the system is unable to make use of certain areas of available main memory. Fixed-partition multiprogramming suffers from internal fragmentation, which occurs when the size of a process's memory and data is smaller than the partition in which the process executes.

Variable-partition multiprogramming allows a process to occupy only as much space as needed (up to the amount of available main memory). Waste does not become obvious until processes finish and leave holes in main memory, resulting in external fragmentation. The system can take measures to reduce some of its external fragmentation by implementing a free memory list to coalesce holes, or perform memory compaction.

The system's memory placement strategy determines where in main memory to place incoming programs and data. The first-fit strategy places an incoming job in main memory in the first available hole that is large enough to hold it. The best- I fit strategy places the job in the hole in main memory in which it fits most tightly and which leaves the smallest amount of unused space. The worst-fit strategy places the job in main memory in the hole in which it fits worst (i.e., in the largest possible hole). A variation of the first-fit, called the next-fir strategy, begins each search for an available hole at the point where the previous search ended.

Memory swapping is a technique in which a process does not necessarily remain in main memory throughout its execution. When a process in memory cannot execute, the system swaps (or rolls) out the old process and swaps (or rolls) in the next process. More sophisticated swapping systems have been developed that allow several processes to remain in main memory at once. In these systems, the system swaps a process out only when an incoming process needs that memory space With a sufficient amount of main memory, these system greatly reduce the time spent swapping.

Key Terms

anticipatory fetch strategy—Method of bringing pages into main memory before they are requested so they will be immediately available when they are requested. This is accomplished by predicting which operations a program will perform next.

base register—Register containing the lowest memory address a process may reference.

best-fit memory placement strategy—Memory placement strategy that places an incoming job in the smallest hole in memory that can hold the job.

boundary register—Register for single-user operating systems that was used for memory protection by separating user memory space from kernel memory space.

burping the memory—See memory compaction.

cache memory—Small, expensive, high-speed memory that holds copies of programs and data to decrease memory access times.

coalescing memory holes—Process of merging adjacent holes in memory in variable partition multiprogramming systems. This helps create the largest possible holes available for incoming programs and data.

contiguous memory allocation—Method of assigning memory such that all of the addresses in the process's entire address space are adjacent to one another.

demand fetch strategy—Method of bringing program parts or data into main memory as they are requested by a process

executive mode—Protected mode in which a processor can execute operating system instructions on behalf of a user (also called kernel mode).

external fragmentation—Phenomenon in variable-partition memory systems in which there are holes distributed throughout memory that are too small to hold a process.

fetch strategy—Method of determining when to obtain the next piece of program or data for transfer from secondary storage to main memory.

first-fit memory placement strategy—Memory placement strategy that places an incoming process in the first hole that is large enough to hold it.

fixed-partition multiprogramming—Memory organization that divides main memory into a number of fixed-size partitions, each holding a single job.

fragmentation (of main memory)—Phenomenon wherein a system is unable to make use of certain areas of available main memory.

free memory list — Operating system data structure that points to available holes in memory.

garbage collection — See memory compaction.

hole - An unused area of memory in a variable-partition multiprogramming system.

input/output control system (IOCS)—Precursor to modern operating systems that provided programmers with a basic set of functions to perform I/O.

intensive resource management—Notion of devoting substantial resources to managing other resources to improve overall utilization.

internal fragmentation—Phenomenon in fixed-partition multiprogramming systems in which there holes occur when the size of a process's memory and data is smaller than the partition in which the process executes.

job control language—Commands interpreted by a job stream processor that define and facilitate the setup of the next job in a single-stream batch-processing system.

job stream processor—Entity in single-stream batch-processing systems that controls the transition between jobs.

job-to-job transition—Time during which jobs cannot execute in single-stream batch-processing systems while one job is purged from the system and the next job is loaded and prepared for execution.

limit register—Register used in fixed-partition multiprogramming systems to mark where a process's memory partition ends.

memory compaction—Relocating all partitions in a variable-partition multiprogramming system to one end of main memory to create the largest possible memory hole.

memory hierarchy—Model that classifies memory into levels corresponding to speed, size and cost.

memory management strategy—Specification of how a particular memory organization performs operations such as fetching, placing and replacing memory.

memory manager—Component of an operating system that implements the system's memory organization and memory management strategies.

memory organization—Manner in which the system views main memory, addressing concerns such as how many processes exist in memory, where to place programs and data in memory and when to replace those pieces with other pieces.

next-fit memory placement strategy—Variation of the first-fit memory placement strategy that begins each search for an available hole at the point where the previous search ended.

noncontiguous memory allocation—Method of memory allocation that divides a program into several, possibly nonadjacent, pieces that the system places throughout main memory.

overlay—Concept created to enable programs larger than main memory to run. Programs are broken into pieces that do not need to exist simultaneously in memory. An overlay contains one such piece of a program.

partition—Portion of main memory allocated to a process in fixed- and variable-partition multiprogramming. Programs are placed into partitions so that the operating system can protect itself from user processes and so that processes are protected from each other.

physical memory—See main memory.

placement strategy (main memory)—Strategy that determines where in the main memory to place incoming programs and data.

replacement strategy (main memory)—Method that a system uses to determine which piece of program or data to displace to accommodate incoming programs or data.

roll — See swap.

setup time—Time required by a system operator and the operating system to prepare the next job to be executed.

single-stream batch-processing system—Batch-processing system that places ready jobs in available partitions from one queue of pending jobs.

single-user contiguous memory allocation system—System in which programs are placed in adjacent memory addresses and the system services only one program at a time.

supervisor call—Request by a user process to the operating system to perform an operation on its behalf (also called a system call).

swap—Method of copying a process's memory contents to secondary storage, removing the process from memory and allocating the freed memory to a new process.

teardown time—Time required by a system operator and the operating system to remove a job from a system after the job has completed.

temporal locality—Property of events that are closely related over time. In memory references, temporal locality occurs when processes reference the same memory locations repeatedly within a short period.

variable-partition multiprogramming—Method of assigning partitions that are the exact size of the job entering the system.

wall clock time—Measure of time as perceived by a user.

worst-fit strategy—Memory placement strategy that places an incoming job in the largest hole in memory.

Exercises

9.1 In hierarchical memory systems, a certain amount of overhead is involved in moving programs and data between the various levels of the hierarchy. Discuss why the benefits derived from such systems justify the overhead involved.

9.2 Why did demand fetching endure as the conventional wisdom for so long? Why are anticipatory fetch strategies receiving so much more attention today than they did decades ago?

9.3 Discuss how memory fragmentation occurs in each of the memory organization schemes presented in this chapter.

9.4 In what circumstances are overlays useful? When may a section of main memory be overlayed? How does overlaying affect program development time? How does overlaying affect program modifiability?

9.5 Discuss the motivations for multiprogramming. What characteristics of programs and machines make multiprogramming desirable? In what circumstances is multiprogramming undesirable?

9.6 You are given a hierarchical memory system consisting of four levels—cache, primary memory, secondary memory and tertiary memory. Assume that programs may be executed in any of the memory levels. Each level consists of an identical amount of memory, and the range of memory addresses in each level is identical. Cache runs programs the fastest, primary memory is ten times slower than the cache, secondary memory is ten times slower than primary memory and tertiary memory is ten times slower than secondary memory. There is only one processor and it may execute only one program at a time.

a. Assume that programs and data may be shuttled from any level to any other level under the operating system's control. The time it takes to transfer items between two particular levels is dependent upon the speed of the lowest (and slowest) level involved in the transfer. Why might the operating system choose to shuttle a program from cache directly to secondary memory, thus bypassing primary memory? Why would items be shuttled to slower levels of the hierarchy? Why would items be shuttled to faster levels of the hierarchy?

b. The scheme above is somewhat unconventional. It is more common for programs and data to be moved only between adjacent levels of the hierarchy. Give several arguments against allowing transfers directly from the cache to any level other than primary memory.

9.7 As a systems programmer in a large computer installation using a fixed-partition multiprogramming system, you have the task of determining if the current partitioning of the system should be altered.

a. What information would you need to help you make your decision?

b. If you had this information readily available, how would you determine the ideal partitioning?

c. What are the consequences of repartitioning such a system?

9.8 A simple scheme for relocating programs in multiprogramming environments involves the use of a single relocation register. All programs are translated to locations beginning at zero, but every address developed as the program executes is modified by adding to it the contents of the processor's relocation register. Discuss the use and control of the relocation register in variable-partition multiprogramming. How might the relocation register be used in a protection scheme?

9.9 Placement strategies determine where in the main memory incoming programs and data should be loaded. Suppose a job waiting to begin execution has a memory requirement that can be fulfilled immediately. Should the job be loaded and begin execution immediately?

9.10 Charging for resources in multiprogramming system can be complex.

a. In a dedicated system, the user is normally charged for the entire system. Suppose that in a multiprogramming system only one user is currently on the system. Should the user be charged for the entire system?

b. Multiprogramming operating systems generally consume substantial system resources, as they manage multiple-user environments. Should users pay for this overhead, or should it be "absorbed" by the operating system?

c. Most people agree that charges for computer system usage should be fair, but few can define precisely what "fairness" is. Another attribute of charging schemes, but one which is easier to define, is predictability. We want to know that if a job costs a certain amount to run once, running it again in similar circumstances will cost approximately the same amount. Suppose that in a multiprogramming environment we charge by wall clock time, i.e., the total real time involved in running the job from start to completion. Would such a schemes yield predictable charges? Why?

9.11 Discuss the advantages and disadvantages of noncontiguous memory allocation.

9.12 Many designers believe that the operating system shold always be given a "most trusted" status. Some designers fell that even the operating system should be curtailed, particularly

in its ability to reference certain areas of memory. Discuss the pros and cons of allowing the operating system to access the full range of real addresses in a computer system at all times.

9.13 Developments in operating systems have generally occurred in an evolutionary rather than revolutionary fashion. For each of the following transitions, describe the primary motivations that led operating systems designers to produce the new type of system from the old.

a. Single-user dedicated systems to multiprogramming

b. Fixed-partition multiprogramming systems with absolute translation and loading to fixed-partition multiprogramming systems with relocatable translation and loading

c. Fixed-partition multiprogramming to variable-partition multiprogramming

d. Contiguous memory allocation systems to noncontiguous memory allocation systems

e. Single-user dedicated systems with manual job-to-job transition to single-user dedicated systems with single-stream batch-processing systems

9.14 Consider the problem of jobs waiting in a queue until sufficient memory becomes available for them to be loaded and executed. If the queue is a simple first-in-first-out structure, then only the job at the head of the queue may be considered for placement in memory. With a more complex queuing mechanism, it might be possible to examine the entire queue to choose the next job to be loaded and executed. Show how the latter discipline, even though more complex, might yield better throughput than the simple first-in-first-out strategy. What problem could the latter approach suffer from?

9.15 One pessimistic operating systems designer says it really does not matter what memory placement strategy is used. Sooner or later a system achieves steady state and all of the strategies perform similarly. Do you agree? Explain.

9.16 Another pessimistic designer asks why we go to the trouble of defining a strategy with an official-sounding name like first-fit. This designer claims that a first-fit strategy is equivalent to nothing more than random memory placement. Do you agree? Explain.

9.17 Consider a swapping system with several partitions. The absolute version of such a system would require that programs be repeatedly swapped in and out of the same partition. The relocatable version would allow programs to be swapped in and out of any available partitions large enough to hold them, possibly different partitions on successive swaps. Assuming that main memory is many times the size of the average job, discuss the advantages of this multiple-user swapping scheme over the single-user swapping scheme described in the text.

9.18 Sharing procedures and data can reduce the main memory demands of jobs, thus enabling a higher level of multiprogramming. Indicate how a sharing mechanism might be implemented for each of the following schemes. If you feel that sharing is inappropriate for certain schemes, say so and explain why.

a. fixed-partition multiprogramming with absolute translation and loading

b. fixed-partition multiprogramming with relocatable translation and loading

c. variable-partition multiprogramming

d. multiprogramming in a swapping system that enables two jobs to reside in main memory at once, but that multiprograms more than two jobs.

9.19 Much of the discussion in this chapter assumes that main memory is a relatively expensive resource that should be managed intensively. Imagine that main memory eventually becomes so abundant and so inexpensive, that users could essentially have all they want. Discuss the ramifications of such a development on

a. operating system design and memory management strategies

b. user application design.

9.20 In this exercise, you will examine the next-fit strategy and compare it to first-fit.

a. How would the data structure used for implementing next-fit differ from that used for first-fit?

b. What happens in first-fit if the search reaches the highest-addressed block of memory and discovers it is not large enough?

c. What happens in next-fit when the highest-addressed memory block is not large enough?

d. Which strategy uses free memory more uniformly?

e. Which strategy tends to cause small blocks to collect at low memory addresses?

f. Which strategy does a better job keeping large blocks available?

9.21 *(Fifty-Percent Rule)* The more mathematically inclined student may want to attempt to prove the "fifty-percent rule" developed by Knuth.[40, 41] The rule states that at steady state a variable-partition multiprogramming system will tend to have approximately half as many holes as there are occupied memory blocks. It assumes that the vast majority of "fits" are not exact (so that fits tend *not* to reduce the number of holes).

9.22 A variable-partition multiprogramming system uses a free memory list to track available memory. The current list

contains entries of 150KB, 360KB, 400KB, 625KB, and 200KB. The system receives requests for 215KB, 171KB, 86KB, and 481KB. in that order. Describe the final contents of the free memory list if the system used each of the following memory placement strategies.

a. best-fit
b. first-fit
c. worst-fit
d. next-fit

9.23 This exercise analyzes the worst-fit and best-fit strategies discussed in Section 9.9, Variable-Partition Multiprogramming, with various data structures.

a. Discuss the efficiency of worst-fit when locating the appropriate memory hole in a free memory list stored in the following data structures.
   i. unsorted array
   ii. array sorted by size of hole
   iii. unsorted linked list
   iv. linked list sorted by size of hole
   v. binary tree
   vi. balanced binary tree
b. Discuss the efficiency of best-fit when locating the appropriate memory hole in a free memory list stored in the following data structures.
   i. unsorted array
   ii. array sorted by size of hole
   iii. unsorted linked list
   iv. linked list sorted by size of hole
   v. binary tree
   vi. balanced binary tree
c. Comparing your results from parts a and b, it might appear that one algorithm is more efficient than the other. Why is this an invalid conclusion?

Suggested Projects

9.24 Prepare a research paper on the use of real memory management in real-time systems.

9.25 Prepare a research paper describing variations on the traditional memory hierarchy.

Suggested Simulations

9.26 Develop a simulation program to investigate the relative effectiveness of the first-fit, best-fit, and worst-fit memory placement strategies. Your program should measure memory utilization and average turnaround time for the various memory placement strategies. Assume a real memory system with 1GB capacity, of which 300MB is reserved for the operating system. New processes arrive at random intervals between 1 and 10 minutes (in multiples of 1 minute), the sizes of the processes are random between 50MB and 300MB in multiples of 10MB, and the durations range from 5 to 60 minutes in multiples of 5 minutes in units of 1 minute. Your program should simulate each strategy over a long enough interval to achieve steady-state operation.

a. Discuss the relative difficulties of implementing each strategy.
b. Indicate any significant performance differences you observed.
c. Vary the job arrival rates and job size distributions and observe the results for each strategy.
d. Based on your observations, state which strategy you would choose if you were actually implementing a physical memory management system.
e. Based on your observations, suggest some other memory placement strategy that you think would be more effective than the ones you have investigated here Simulate its behavior. What results do you observe?

For each of your simulations, assume that the time it takes to make the memory placement decision is negligible.

9.27 Simulate a system that initially contains 100MB of free memory, in which processes frequently enter and exit the system. The arriving processes are between 2MB and 35MB in size, and their execution times vary between 2 and 5 seconds. Vary the rate of arrival between two processes per second and one process every 5 seconds. Evaluate memory utilization and throughput when using the first-fit, best-fit and worst-fit strategies. Keeping each placement strategy constant, vary the process scheduling algorithm to evaluate its effect on throughout and average wait times. Incorporate scheduling algorithms from Chapter 8, such as FIFO and SPF, and evaluate other algorithms, such as smallest-process-first and largest-process-first. Be sure to determine which, if any, of these algorithms can suffer from indefinite postponement.

Recommended Reading

As a result of the widespread usage of virtual memory, many of the techniques in this chapter are no longer discussed in the literature. Belady et al.[42] provided a thorough treatment of the history of memory management in IBM systems. Mitra[43] and Pohm and Smay[44] give a description of the memory hierarchy that has become the standard in architectures for decades. Bayes[45] and Stephenson[46] presented the primary strategies for memory allocation and placement. Denning[47] presented the pitfalls of single-user contiguous memory allocation, while Knight[48] and Coffman and Ryan[49] investigated the implementation of partitioned memory. Memory placement algorithms were presented by Knuth,[50] Stephenson[51] and Oldehoeft and Allan;[52] memory swapping concepts, which are vital to virtual memory, are presented in further detail in the next chapter.

Recent research has focused on optimizing the use of the memory hierarchy. Several memory management strategies have focused on a faster and more expensive resource, cache.[53] The literature also reports application-specific tuning of the memory architecture to improve performance.[54, 55]

Embedded systems are designed differently than general-purpose systems. Whereas general-purpose systems will provide enough resources to perform a wide variety of tasks concurrently, embedded systems provide the bare minimum necessary to meet their design goals. Due to size and cost considerations, resources in embedded systems are much more scarce than in general-purpose systems. The *ACM Transactions on Embedded Computing Systems,* in their November 2002 and February 2003 issues, explored a great deal of research on the topic of memory management in embedded systems.[56] The bibliography for this chapter is located on our Web site at www.deitel.com/books/os3e/Bibliography.pdf.

Works Cited

1. Belady, L. A.; Parmelee, R. P.; and C. A. Scalzi, "The IBM History of Memory Management Technology," *IBM Journal of Research and Development,* Vol. 25, No. 5, September 1981, pp. 491-503.
2. Belady, L. A.; Parmelee, R. P.; and C. A. Scalzi, "The IBM History of Memory Management Technology," *IBM Journal of Research and Development,* Vol. 25, No. 5, September 1981, pp. 491-503. "How Much Memory Does My Software Need?" <www.crucial.com/1ibrary/softwareguide.asp>, viewed luly 15,2003.
3. Belady, L. A.; Parmelee, R. P.; and C. A. Scalzi, "The IBM History of Memory Management Technology," *IBM Journal of Research and Development,* Vol. 25, No. 5, September 1981, pp. 491-503. "How Much Memory Does My Software Need?" <www.crucial.com/1ibrary/softwareguide.asp>, viewed July 15,2003.
4. "Microsoft/Windows Timeline," <www.technicalminded.com/windowstimeline.htm>, last modified February 16,2003.
5. "Windows Version History," <support.microsoft.com/default.aspx?scid=http://support.mi crosoft.com:80/support/kb/articles/Q32/9/05.asp&NoWebContent=l>, last reviewed September 20,1999.
6. Anand Lai Shimpi, "Western Digital Raptor Preview: 10,000 RPM and Serial ATA," AnandTech.com, March 7, 2003, <www.anandtech.com/storage/showdoc.html?i=1795&p=9>.
7. Todd Tokubo, "Technology Guide: DDR RAM," GamePC.com, September 14,2000, <www.gamepc.com/1abs/viewcontent.asp?id=ddrguide&page=3>.
8. Baskett, E; Broune, J. C; and W M. Raike, "The Management of a Multi-level Non-paged Memory System," *Proceedings Spring Joint Computer Conference,* 1970, pp. 459^65.
9. Lin, Y. S., and R. L. Mattson, "Cost-Performance Evaluation of Memory Hierarchies," *IEEE Transactions Magazine,* MAG-8, No. 3, September 1972, p. 390.
10. Mitra, D., "Some Aspects of Hierarchical Memory Systems," *JACM,* Vol. 21, No. 1, January 1974, p. 54.
11. Pohm, A. V, and T. A. Smay, "Computer Memory Systems," *IEEE Computer,* October 1981, pp. 93-110.
12. Pohm, A. V., and T. A. Smay, "Computer Memory Systems," *IEEE Computer,* October 1981, pp. 93-110.
13. Smith, A. J., "Cache Memories," *ACM Computing Surveys,* Vol. 14, No. 3, September 1982, pp. 473-530.
14. Bays, C, "A Comparison of Next-fit, First-fit, and Best-fit," *Communications of the ACM,* Vol. 20, No. 3, March 1977, pp. 191-192.
15. Stephenson, C. X, "Fast Fits: New Methods for Dynamic Storage Allocation," *Proceedings of the 9th Symposium on Operating Systems Principles,* ACM, Vol. 17, No. 5, October 1983, pp. 30-32.
16. Belady, L. A.; R. P. Parmelee; and C. A. Scalzi, "The IBM History of Memory Management Technology," *IBM Journal of Research and Development,* Vol. 25, No. 5, September 1981, pp. 491-503.
17. Feiertag, R. J. and Organick, E. I., "The Multics Input/Output System," *ACM Symposium on Operating Systems Principles,* 1971, pp. 35-41.
18. Denning, P. J., "Third Generation Computer Systems," *ACM Computing Surveys,* Vol. 3, No. 4, December 1971, pp. 175-216.
19. Denning, P. X, "Third Generation Computer Systems," *ACM Computing Surveys, Vol.* 3, No. 4, December 1971, pp. 175-216.
20. Dennis, X B., "A Multiuser Computation Facility for Education and Research," *Communications of the ACM,* Vol. 7, No. 9, September 1964, pp. 521-529.

21. Belady, L. A.; Parmelee, R. P.; and C. A. Scalzi, "The IBM History of Memory Management Technology," *IBM Journal of Research and Development,* Vol. 25, No. 5, September 1981, pp. 491-503.
22. Knight, D. C, "An Algorithm for Scheduling Storage on a Non-Paged Computer," *Computer Journal,* Vol. 11, No. 1, February 1968, pp. 17-21.
23. Denning, P., "Virtual Memory", *ACM Computing Surveys,* Vol. 2, No. 3, September 1970, pp. 153-189.
24. Randell, B., "A Note on Storage Fragmentation and Program Segmentation," *Communications of the ACM,* Vol. 12, No. 7, July 1969, pp. 365-372.
25. Knight, D. C, "An Algorithm for Scheduling Storage on a Non-Paged Computer," *Computer Journal,* Vol. 11, No. 1, February 1968, pp. 17-21.
26. Coffman, E. G., and T. A. Ryan, "A Study of Storage Partitioning Using a Mathematical Model of Locality," *Communications of the ACM,* Vol. 15, No. 3, March 1972, pp. 185-190.
27. Belady, L. A.; Parmelee, R. P.; and C. A. Scalzi, "The IBM History of Memory Management Technology," *IBM Journal of Research and Development,* Vol. 25, No. 5, September 1981, pp. 491-503.
28. Randell, B., "A Note on Storage Fragmentation and Program Segmentation," *Communications of the ACM,* Vol. 12, No. 7, July 1969, pp. 365-372.
29. Margolin, B. H.; Parmelee, R. P.; and M. Schatzoff, "Analysis of Free-Storage Algorithms," *IBM Systems Journal,* Vol. 10, No. 4, 1971, pp. 283-304.
30. Bozman, G.; Buco, W.; Daly, T. P.; and W. H. Tetzlaff, "Analysis of Free-Storage Algorithms—Revisited," *IBM Systems Journal,* Vol. 23, No. 1,1984, pp. 44-66.
31. Baskett, F; Broune, J. C; and W. M. Raike, "The Management of a Multi-level Non-paged Memory System," *Proceedings Spring Joint Computer Conference,* 1970, pp. 459-165.
32. Davies, D. J. M., "Memory Occupancy Patterns in Garbage Collection Systems," *Communications of the ACM,* Vol. 27, No. 8, August 1984, pp. 819-825.
33. Knuth, D. E., *The Art of Computer Programming: Fundamental Algorithms,* Vol. 1,2nd ed., Reading, MA: Addison-Wesley, 1973.
34. Stephenson, C. J., "Fast Fits: New Methods for Dynamic Storage Allocation," *Proceedings of the 9th Symposium on Operating Systems Principles, ACM,* Vol. 17, No. 5, October 1983, pp. 30-32.
35. Oldehoeft, R. R., and S. J. Allan, "Adaptive Exact-fit Storage Management," *Communications of the ACM,* Vol. 28, No. 5, May 1985, pp. 506-511.
36. Shore, J., "On the External Storage Fragmentation Produced by First-fit and Best-fit Allocation Strategies," *Communications of the ACM,* Vol. 18, No. 8, August 1975, pp. 433-440.
37. Bays, C, "A Comparison of Next-fit, First-fit, and Best-fit," *Communications of the ACM,Vol.* 20, No. 3, March 1977, pp. 191-192.
38. Ritchie, D. M., and K. T. Thompson, "The UNIX Time-sharing System," *Communications of the ACM, Vol.* 17, No. 7, July 1974, pp. 365-375.
39. Belady, L. A.; Parmelee, R. P.; and C. A. Scalzi, "The IBM History of Memory Management Technology," *IBM Journal of Research and Development,* Vol. 25, No. 5, September 1981, pp. 491-503.
40. Knuth, D. E., *The Art of Computer Programming: Fundamental Algorithms,* Vol. 1,2nd ed., Reading, MA: Addison-Wesley, 1973.
41. Shore, J. E., "Anomalous Behavior of the Fifty-Percent Rule." *Communications of the ACM,* Vol. 20, No. 11, November 1977. pp. 812-820.
42. Belady, L. A.; Parmelee, R. P.; and C. A. Scalzi, "The IBM History of Memory Management Technology," *IBM Journal of Research and Development,* Vol. 25, No. 5, September 1981, pp. 491-503.
43. Mitra, D, "Some Aspects of Hierarchical Memory Systems." *JACM,* Vol. 21, No. 1, January 1974, p. 54.
44. Pohm, A. V, and T A. Smay, "Computer Memory Systems." *IEEE Computer,* October 1981, pp. 93-110.
45. Bays, C, "A Comparison of Next-fit, First-fit, and Best-fit," *Communications of the ACM,* Vol. 20, No. 3, March 1977, pp. 191-192.
46. Stephenson, C. J., "Fast Fits: New Methods for Dynamic Storage Allocation," *Proceedings of the 9th Symposium on Operating Systems Principles,* ACM, Vol. 17, No. 5, October 1983, pp. 30-32.
47. Denning, P. X, "Third Generation Computer Systems," *ACM Computing Surveys,* Vol. 3, No. 4, December 1971, pp. 175-216.
48. Knight, D. C, "An Algorithm for Scheduling Storage on a Non-Paged Computer," *Computer Journal,* Vol. 11, No. 1, Februar. 1968, pp. 17-21.
49. Coffman, E. G., and T. A. Ryan, "A Study of Storage Partitioning Using a Mathematical Model of Locality," *Communications of the ACM,* Vol. 15, No. 3, March 1972, pp. 185-190.
50. Knuth, D. E., *The Art of Computer Programming: Fundamental Algorithms,* Vol. 1,2nd ed., Reading, MA: Addison-Wesley, 1973.
51. Stephenson, C. J., "Fast Fits: New Methods for Dynamic Storage Allocation," *Proceedings of the 9th Symposium on Operating Systems Principles, ACM,* Vol. 17, No. 5, October 1983, pp. 30-32.
52. Oldehoeft, R. R., and S. J. Allan, "Adaptive Exact-fit Storage Management," *Communications of the ACM,* Vol. 28, No. 5, May 1985, pp. 506-511.
53. Luk, C. and Mowry, T C, "Architectural and Compiler Suppor. for Effective Instruction Prefetching: A Cooperative Approach." *ACM Transactions on Computer Systems (TOCS),* Vol. 19 No 1, February 2001.
54. Mellor-Crummey, J.; Whalley, D; and K. Kennedy, "Improving Memory Hierarchy Performance for Irregular Applications." *Proceedings of the 13th International Conference on Supercomputing,* May 1999.
55. Mellor-Crummey, X; Fowler, R.; and D. Whalley, "Tools for Application-oriented Performance Tuning," *Proceedings of the 15th International Conference on Supercomputing,* June 2001.
56. Jacob, B. and S. Bhattacharyya, "Introduction to the Two Special Issues on Memory," *ACM Transactions on Embedded Computing Systems (TECS),* February 2003, Vol. 2, No. 1.

The fancy is indeed no other than a mode of memory emancipated from the order of time and space.

— Samuel Taylor Coleridge—

Lead me from the unreal to the real!

— *The Upanishads*—

O happy fault, which has deserved to have such and so mighty a Redeemer.

—The Missal—*The Book of Common Prayer*—

But every page having an ample marge,
And every marge enclosing in the midst
A square of text that looks a little blot.

—Alfred, Lord Tennyson—

Addresses are given to us to conceal our whereabouts.

-Saki(H.H.Munro)-

Chapter 10

Virtual Memory Organization

Objectives

After reading this chapter, you should understand:

- *the concept of virtual memory.*
- *paged virtual memory systems.*
- *segmented virtual memory systems.*
- *combined segmentation/paging virtual memory systems.*
- *sharing and protection in virtual memory systems.*
- *the hardware that makes virtual memory systems feasible.*
- *the IA-32 Intel architecture virtual memory implementation.*

Chapter Outline

10.1 Introduction

Mini Case Study: Atlas

Operating Systems Thinking: Virtualization

Biographical Note: Peter Denning

10.2 Virtual Memory: Basic Concepts

Anecdote: Virtual Memory Unnecessary

10.3 Block Mapping

10.4 Paging

10.4.1 Paging Address Translation by Direct Mapping

10.4.2 Paging Address Translation by Associative Mapping

10.4.3 Paging Address Translation with Direct/Associative Mapping

Operating Systems Thinking: Empirical Results: Locality-Based Heuristics

10.4.4 Multilevel Page Tables

10.4.5 Inverted Page Tables

10.4.6 Sharing in a Paging System

Operating Systems Thinking: Lazy Allocation

10.5 Segmentation

10.5.1 Segmentation Address Translation by Direct Mapping

10.5.2 Sharing in a Segmentation System

10.5.3 Protection and Access Control in Segmentation Systems

10.6 Segmentation/Paging Systems

10.6.1 Dynamic Address Translation in a Segmentation/Paging System

10.6.2 Sharing and Protection in a Segmentation/Paging System

10.7 Case Study: IA-32 Intel Architecture Virtual Memory

Mini Case Study: IBM Mainframe Operating Systems

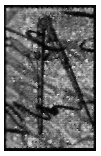
Mini Case Study: Early History of the VM Operating System

Web Resources | Summary | Key Terms | Exercises | Recommended Reading | Works Cited

10.1 Introduction

In the preceding chapter, we discussed basic memory management techniques, each of which ultimately must contend with limited memory space. One solution is larger main memories; however, it is often prohibitively expensive to build systems with sufficiently large fast-access memory. Another solution would be to create the illusion that more memory exists. This is a fundamental idea behind **virtual memory,**[1, 2, 3] which first appeared in the Atlas computer system constructed at the University of Manchester in England in 1960 (see the Mini Case Study, Atlas).[4, 5, 6, 7] Figure 10.1 shows how memory organizations have evolved from single-user real memory systems to multiuser, segmentation/paging virtual memory systems.

This chapter describes how a system implements virtual memory (see the Operating Systems Thinking feature, Virtualization). Specifically, we present the techniques the operating system and the hardware use to convert virtual addresses to physical addresses; and we discuss the two most common noncontiguous alloca-



Mini Case Study

Atlas

The Atlas project began in 1956 at the University of Manchester, England. The project, originally called the MUSE (short for micro-SEcond; "micro" is represented by the Greek letter "mu"), was launched to compete with the high-performance computers being produced by the United States.[8] In 1959, Ferranti Ltd. joined the project as a corporate sponsor, and the first Atlas computer was completed at the end of 1962.[9] Though the Atlas computer was produced for only about 10 years, it made significant contributions computing technology. Many of these appeared in the computer's operating system, the Atlas Supervisor.

The Atlas Supervisor operating system was the first to implement virtual memory.[10] Setting a new standard for job throughput, the Atlas Supervisor was able to execute 16 jobs at once.[11] It also provided its operators with detailed runtime statistics, whereas most other systems required operators to determine them manually. The Atlas Supervisor was also one of the earliest computers to design detailed hardware guidelines in early stages of development to simply systems programming.[12]

Although Atlas was the most powerful computer when it was completed in 1962, it was mostly ignored by the industry.[13] Only three Atlas computers were ever sold, the last in 1965, only three years after its completion.[14] A contributing factor to Atlas's short lifetime was that its programs had to be written in Atlas Autocode, a language much like Algol 60, but not accepted by most programmers.[15] Atlas's designers were eventually forced to implement more popular languages such as Fortran.[16] Another reason for Atlas's quick demise was that limited resources were devoted to developing it. A year after Atlas's release, ICT bought Ferranti's computer projects. ICT discontinued Atlas in favor of its own series of computers known as the 1900s.[17]

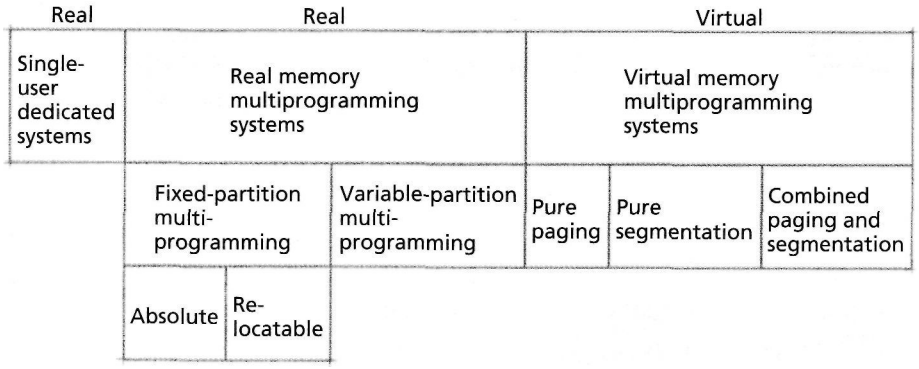


Figure 10.1 | *Evolution of memory organizations.*

tion techniques—paging and segmentation. Address translation and noncontiguous allocation enable virtual memory systems to create the illusion of a larger memory and to increase the degree of multiprogramming. In the next chapter we focus on how a system manages the processes in virtual memory to optimize performance. Much of the information is based on Denning's survey on virtual memory (see the Biographical Note, Peter Denning).[18]

Operating Systems Thinking

Virtualization

We will see many examples in this book of how software can be used to make resources appear different than they really are. This is called virtualization. We will study virtual memories that appear to be far larger than the physical memory that is installed on the underlying computer. We will study virtual machines, which create the illusion that the computer being used to execute applications is really quite different from the underlying hardware. The Java virtual machine enables programmers to develop portable applications that will run on different computers running different operating systems (and hardware). Virtualization is yet another way of using abundant processor power to provide intriguing benefits to computer users. As computers become more complex, virtualization techniques can help hide that complexity from users, who instead see a simpler, easier-to-use virtual machine defined by the operating system. Virtualization is common in distributed systems, which we cover in detail in Chapters 17 and 18. Distributed operating systems create the illusion of there being a single large machine when in fact massive numbers of computers and other resources are interconnected in complex ways across networks like the Internet.

At the end of this chapter (just before the Web Resources section), you will find two operating systems Mini Case Studies—IBM Mainframe Operating Systems and Early History of the VM Operating System. As you finish reading this chapter, you will have studied the evolution of both real memory and virtual memory organizations. The history of IBM mainframe operating systems closely follows this evolution.

Self Review

1. Give an example of when it might be inefficient to load an entire program into memory before running it.
2. Why is increasing the size of main memory an insufficient solution to the problem of limited memory space?

Ans: **1)** Many programs have error-processing functions that are rarely, if ever, used. Loading these functions into memory reduces space available to processes. **2)** Purchasing additional main memory is not always economically feasible. As we will see, a better solution is to create the illusion that the system contains more memory than the process will ever need.

Biographical Note

Peter Denning

Peter J. Denning is best known for his work on virtual memory in the 1960s. One of his most important papers, "The Working Set Model for Program Behavior," was published in 1968, the same year he received his Ph.D. in Computer Science from MIT.[19] This paper introduced the concept of working sets (discussed in Section 11.7, Working Set Model) for use in virtual memory management and explained why the use of working sets leads to a page-replacement could improve performance over other replacement schemes.[20] Denning's theory is based on observations that programs spend most of their time using only a small subset of their virtual memory pages, which he called the working set. A program's working set changes from time to time as program execution moves along its phases. He proposed that a program will run efficiently if its current working set is kept in main memory at all times.

Denning has since taught at Princeton University, founded the Research Institute for Advanced Computer Science at the NASA Ames Research Center and led the computer science departments of Purdue, George Mason University and now the Naval Postgraduate School. He has published hundreds of articles and several books on computer science topics and was president of the ACM from 1980-1982.[21]

Denning has made significant contributions to computer science education. He won several teaching awards, managed the creation of the ACM Digital Library (an online collection of articles from ACM journals) and chaired the ACM Education Board which publishes the ACM curriculum.[22] He has also been recognized for his efforts to educate the nontechnical community about computer science topics.[23]

10.2 Virtual Memory: Basic Concepts

As we discussed in the previous section, virtual memory systems provide processes with the illusion that they have more memory than is built into the computer (see the Anecdote, Virtual Memory Unnecessary). Thus, there are two types of addresses in virtual memory systems: those referenced by processes and those available in main memory. The addresses that processes reference are called **virtual addresses.** Those available in main memory are called **physical** (or **real) addresses.** *[Note:* In this chapter we use the terms "physical address," "real address" and "main memory address" interchangeably]

Whenever a process accesses a virtual address, the system must translate it to a real address. This happens so often that using a general-purpose processor to perform such translations would severely degrade system performance. Thus, virtual memory systems contain special-purpose hardware called the **memory management unit (MMU)** that quickly maps virtual addresses to real addresses.

A process's **virtual address space,** *V,* is the range of virtual addresses that the process may reference. The range of real addresses available on a particular computer system is called that computer's **real address space,** *R.* The number of addresses in *V* is denoted |*V*|, and the number of addresses in *R* is denoted |*R*|. In virtual memory systems, it is normally the case that |V| >> *R*\ (i.e., the virtual address space is much larger than the real address space).

If we are to permit a user's virtual address space to be larger than its real address space, we must provide a means for retaining programs and data in a large auxiliary storage. A system normally accomplishes this goal by employing a two-level storage scheme (Fig. 10.2). One level is the main memory (and caches) in which in which instructions and data must reside to be accessed by a processor running a process. The other level is secondary storage, which consists of large-capacity

Anecdote

Virtual Memory Unnecessary

When the concept of virtual memory was first introduced, designers mused that virtual memory systems would never be needed if computers could be built with a million words of memory. That was thought to be absurd, so work proceeded on building virtual memory systems. Many of today's desktop systems have a thousand times that much main memory and the vast majority of these systems employ virtual memory.

Lesson to operating systems designers: Take Moore's Law seriously. Hardware capabilities will continue to improve at an exponential pace and software requirements will increase as fast.

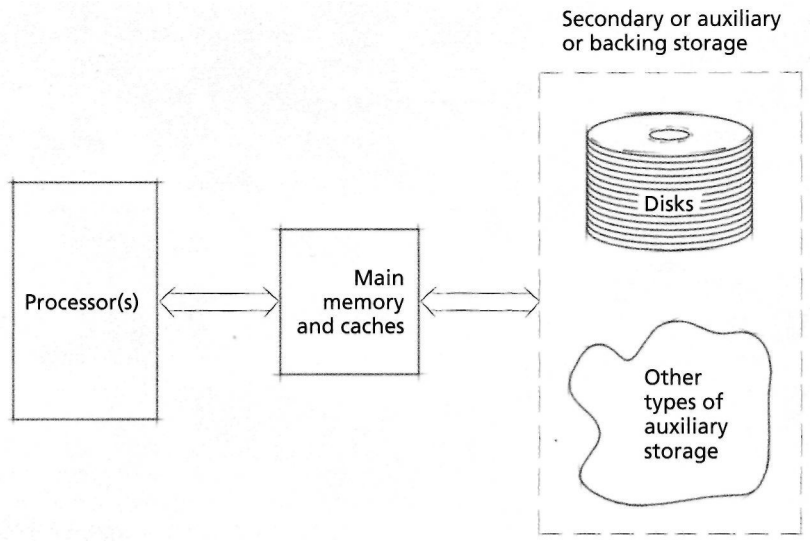


Figure 10.2 | *Two-level storage.*

storage media (typically disks) capable of holding the programs and data that cannot all fit into the limited main memory.

When the system is ready to run a process, the system loads the process's code and data from secondary storage into main memory. Only a small portion of these needs to be in main memory at once for the process to execute. Figure 10.3 illustrates a two-level storage system in which items from various processes' virtual memory spaces have been placed in main memory.

A key to implementing virtual memory systems is how to map virtual addresses to physical addresses. Because processes reference only virtual addresses, but must execute in main memory, the system must map (i.e., translate) virtual addresses to physical addresses as processes execute (Fig. 10.4). The system must perform the translation quickly, otherwise the performance of the computer system would degrade, nullifying the gains associated with virtual memory.

Dynamic address translation (DAT) mechanisms convert virtual addresses to physical addresses during execution. Systems that use dynamic address translation exhibit the property that the contiguous addresses in a process's virtual address space need not be contiguous in physical memory—this is called **artificial contiguity** (Fig. 10.5).[24] Dynamic address translation and artificial contiguity free the programmer from concerns about memory placement (e.g., the programmer need not create overlays to ensure the system can execute the program). The programmer can concentrate on algorithm efficiency and program structure, rather than on the underlying hardware structure. The computer is (or can be) viewed in a logical sense as an implementor of algorithms rather than in a physical sense as a device with unique characteristics, some of which may impede the program development process.

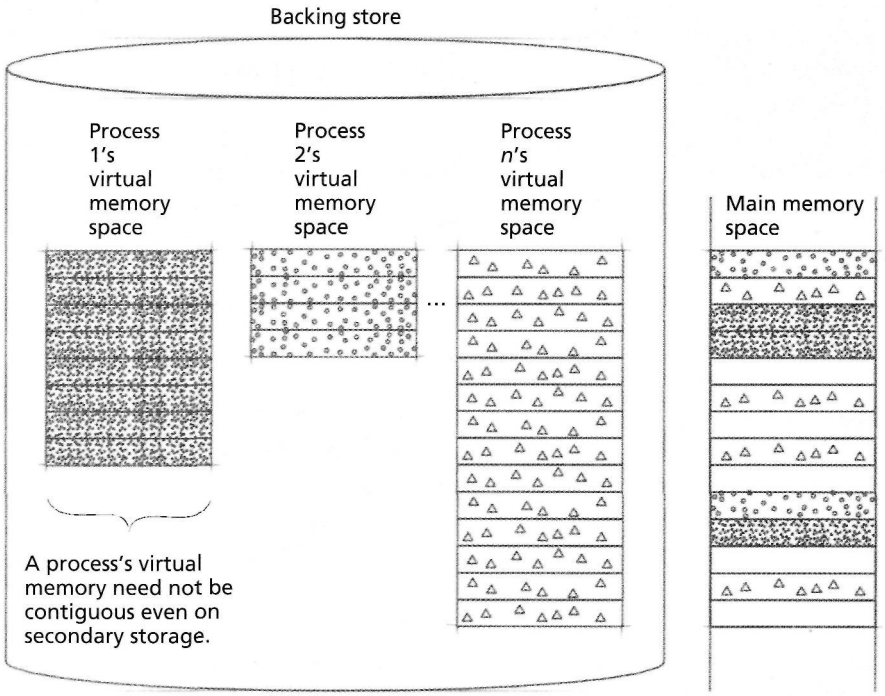


Figure 10.3 | *Pieces of address spaces exist in memory and in secondary storage.*

Self Review

1. Explain the difference between a process's virtual address space and the system's physical address space.
2. Explain the appeal of artificial contiguity.

Ans: **1)** A process's virtual address space refers to the set of addresses a process may reference to access memory when running on a virtual memory system. Processes do not see physical addresses, which locate physical locations in main memory. **2)** Artificial contiguity simplifies programming by enabling a process to reference its memory as if it were contiguous, even though its data and instructions may be scattered throughout main memory.

10.3 Block Mapping

Dynamic address translation mechanisms must maintain **address translation maps** indicating which regions of a process's virtual address space, V, are currently in main memory and where they are located. If this mapping had to contain entries for every address in V, then the mapping information would require more space than is available in main memory, so this is infeasible. In fact, the amount of mapping information must be only a small fraction of main memory, otherwise it would take up too much of the memory needed by the operating system and user processes.

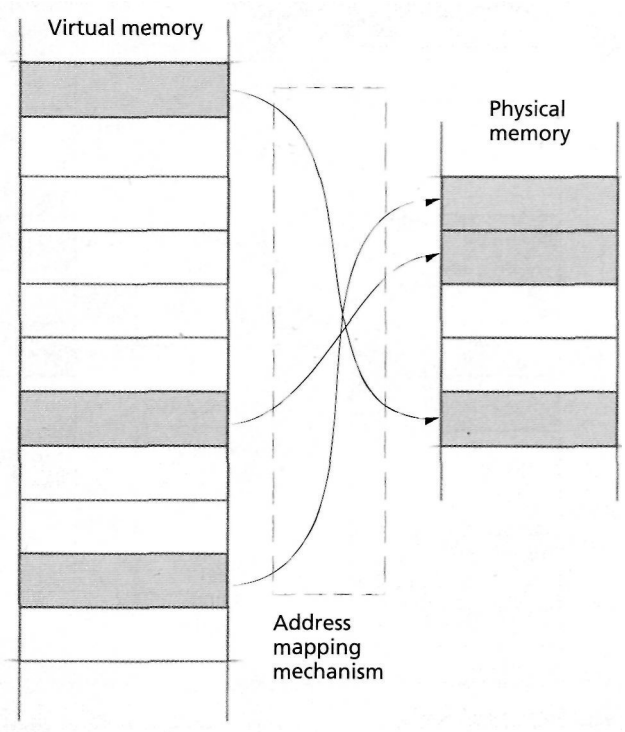


Figure 10.4 | *Mapping virtual addresses to real addresses.*

The most widely implemented solution is to group information in **blocks;** the system then keeps track of where in main memory each virtual memory block has been placed. The larger the average block size, the smaller the amount of mapping information. Larger blocks, however, can lead to internal fragmentation and can take longer to transfer between secondary storage and main memory.

There is some question as to whether the blocks should be all the same size or of different sizes. When blocks are a fixed size (or several fixed sizes), they are called **pages** and the associated virtual memory organization is called **paging.** When blocks may be of different sizes, they are called **segments,** and the associated virtual memory organization is called **segmentation.**[25-26] Some systems combine the two techniques, implementing segments as variable-size blocks composed of fixed-size pages. We discuss paging and segmentation in detail in the following sections.

In a virtual memory system with **block mapping,** the system represents addresses as ordered pairs. To refer to a particular item in the process's virtual address space, a process specifies the block in which the item resides and the **displacement** (or **offset)** of the item from the start of the block (Fig. 10.6). A virtual address, v, is denoted by an ordered pair *(b, d),* where *b* is the block number in which the referenced item resides, and *d* is the displacement from the start of the block.

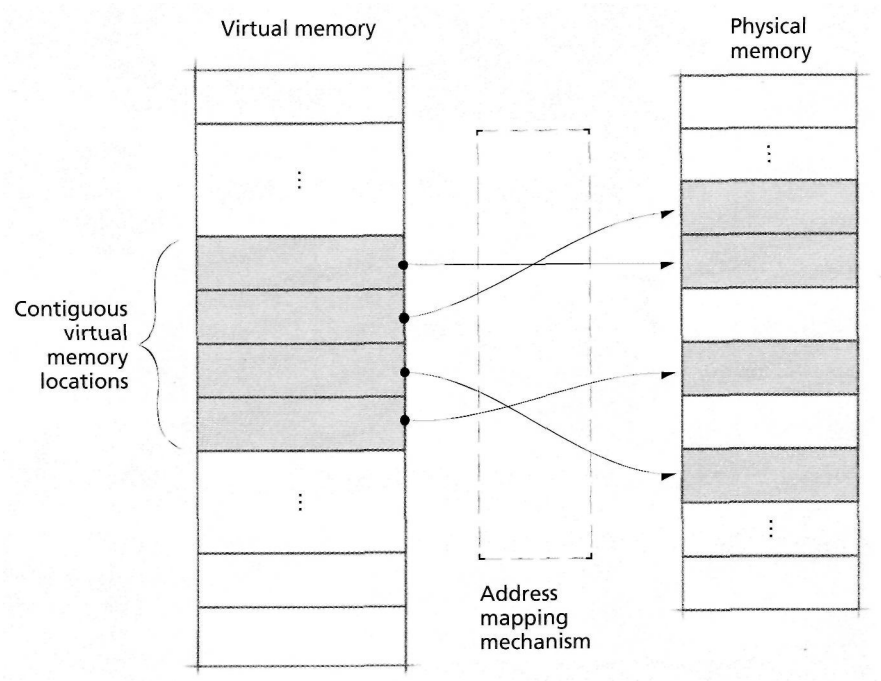


Figure 10.5 | *Artificial contiguity.*

Figure 10. 6 | *Virtual address format in a block mapping system.*

The translation from a virtual memory address v = *(b, d)* to a real memory address, *r,* proceeds as follows (Fig. 10.7). The system maintains in memory a **block map table** for each process. The process's block map table contains one entry for each block of the process, and the entries are kept in sequential order (i.e., the entry for block 0 precedes that for block 1, etc.). At context-switch time, the system determines the real address, *a,* that corresponds to the address in main memory of the new process's block map table. The system loads this address into a high-speed special-purpose register called the **block map table origin register.** During execution, the process references a virtual address *v = (b, d).* The system adds the block number, *b,* to the base address, *a,* of the process's block map table to form the real address of the entry for block *b* in the block map table. *[Note:* For simplicity, we assume that the size of each entry in the block map table is fixed and that each address in memory stores an entry of that size.] This entry contains the address, *b',* for the start of block *b* in main

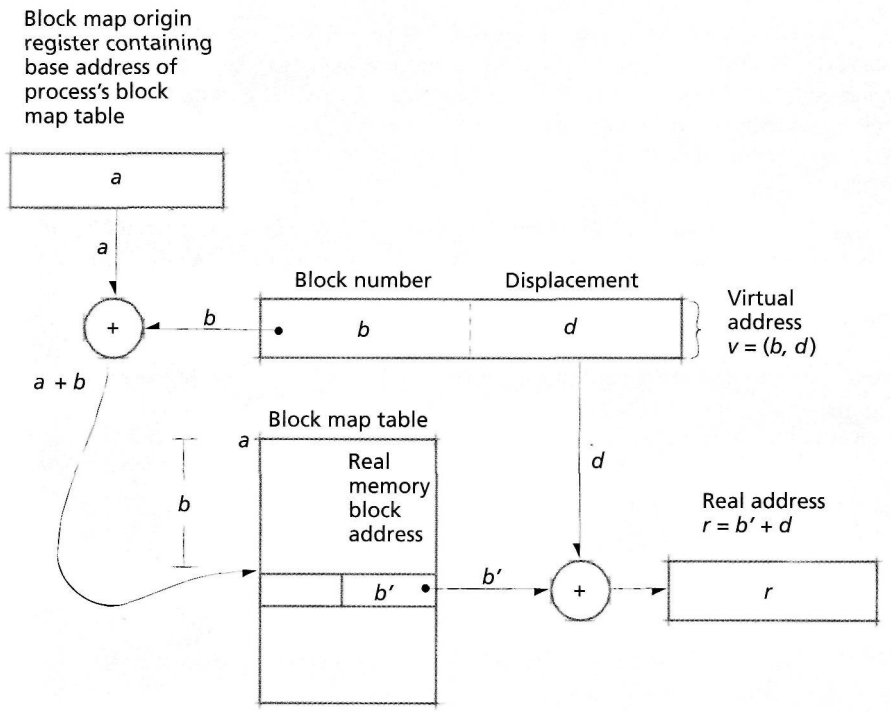


Figure 10.7 | *Virtual address translation with block mapping.*

memory. The system then adds the displacement, *d,* to the block start address, *b′,* to form the desired real address, *r.* Thus, the system computes the real address, *r,* from the virtual address, v = *(b, d),* through the equation $r = b' + d$, where b' is stored in the block map table cell located at real memory address $a + b$.

The block mapping techniques employed in segmentation, paging and combined segmentation/paging systems are all similar to the mapping shown in Fig. 10.7. It is important to note that block mapping is performed dynamically by high-speed, special-purpose hardware as a process runs. If not implemented efficiently, this technique's overhead could cause performance degradation that would negate much of the benefit of using virtual memory. For example, the two additions indicated in Fig. 10.7 must execute much faster than conventional machine-language add instructions. They are performed within the execution of each machine-language instruction for each virtual address reference. If the additions took as long as machine-language adds, then the computer might run at only a small fraction of the speed of a purely real-memory-based computer. Similarly, the block address translation mechanism must access entries in the block mapping table much faster than other information in memory. The system normally places entries from the block mapping table in a high-speed cache to dramatically decrease the time needed to retrieve them.

Self Review

1. Suppose a block mapping system represents a virtual address v = *(b, d)* using 32 bits. If block displacement, *d,* is specified using *n* bits, how many blocks does the virtual address space contain? Discuss how setting $n = 6$, $n = 12$ and $n = 24$ affects memory fragmentation and the overhead incurred by mapping information.
2. Why is the start address of a process's block map table, *a,* placed in a special high-speed register?

Ans: **1)** The system will have 2^{32-n} blocks. If $n = 6$, then the block size would be small and there would be limited internal fragmentation, but the number of blocks would be so large as to make implementation infeasible. If $n = 24$, then the block size would be large and there would be significant internal fragmentation, but the block mapping table would not consume much memory at all. If $n = 12$, the system achieves a balance between moderate internal fragmentation and a reasonably sized block map table. Over the years, $n = 12$ has been quite popular in paging systems, yielding a page size of $2^{12} = 4{,}096$ bytes, as we will see in Chapter 11. Virtual Memory Management. **2)** Placing a in a high-speed register facilitates fast address translation, which is crucial in making the virtual memory implementation feasible.

10.4 Paging

A virtual address in a paging system is an ordered pair *(p, d),* where *p* is the number of the page in virtual memory on which the referenced item resides, and *d* is the displacement within page *p* at which the referenced item is located (Fig. 10.8). A process may run if the page it is currently referencing is in main memory. When the system transfers a page from secondary storage to main memory, it places the page in a main memory block, called a **page frame,** that is the same size as the incoming page. In discussing paging systems in this chapter, we assume that the system uses a single fixed page size. As we will explain in Chapter 11, Virtual Memory Management, it is common for today's systems to provide more than one page size.[27] Page frames begin at physical memory addresses that are integral multiples of the fixed page size, p_s (Fig. 10.9). The system may place an incoming page in any available page frame.

Dynamic address translation under paging proceeds as follows (Fig. 10.10). A running process references a virtual memory address *v - (p,d). A* page mapping mechanism, looks up page *p* in the process's **page map table** (often simply called the **page table)** and determines that page *p* is in page frame *p'.* Note that/?' is a page frame number, not a physical memory address. Assuming page frames are num-

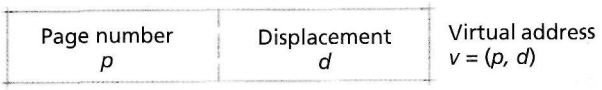


Figure 10.8 | *Virtual address format in a pure paging system.*

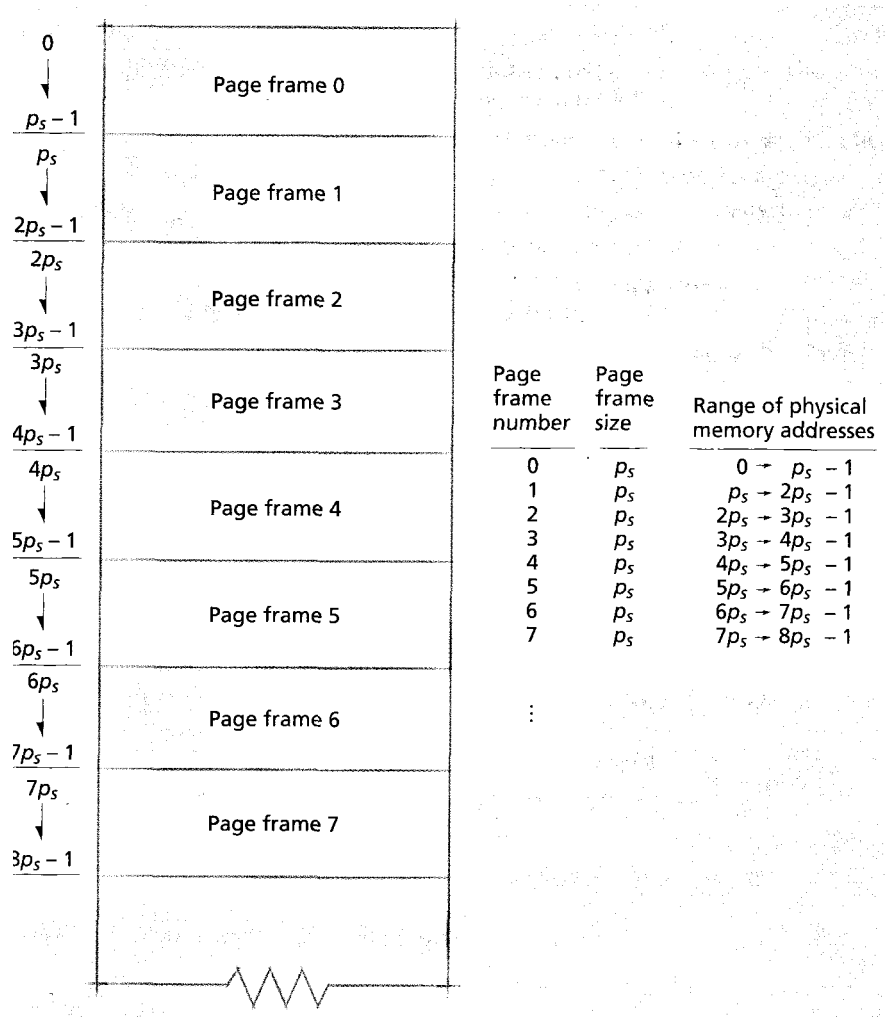


Figure 10.9 | *Main memory divided into page frames.*

bered {0,1, 2,…, *n],* the physical memory address at which page frame p' begins is the product of p' and the fixed page size, $p \times p_s$. The referenced address is formed by adding the displacement, *d,* to the physical memory address at which page frame p' begins. Thus, the real memory address is $r - (p' \times p_s) + d$.

Consider a system using n bits to represent both real and virtual addresses; the page number is represented by the most-significant n-m bits and the displacement is represented by m bits. Each real address can be represented as the ordered pair, $r = (p', d)$, where p' is the page frame number and d is the displacement within page p' at which the referenced item is located. The system can form a real address by concatenating p' and d, which places p' in the most-significant bits of the real mem-